

Degree Programme

Systems Engineering

Major Infotronics

Bachelor's Thesis

Diploma 2026

Basile Gasser

Electric Vehicle Charging Station Management System

Professor

HEI-Vs, Prof. Christopher Métrailler, christopher.metrailler@hevs.ch

Expert

HES-SO Valais Wallis, Samy Francelet, sfrancelet@digital-logic.ch

Submission date

Aug 14, 2026



Filière / Studiengang SYND	Année académique / Studienjahr 2025-26	No TB / Nr. BA IT/2026/9
Mandant / Auftraggeber ☒ HES—SO Valais-Wallis ☐ Industrie ☐ Etablissement partenaire Partnerinstitution	Etudiant·e / Student/in Gasser Basile Professeur·e / Dozent/in Métraiier Christopher	Lieu d'exécution / Ausführungsort ☒ HES—SO Valais-Wallis ☐ Industrie ☐ Etablissement partenaire Partnerinstitution
Travail confidentiel / vertrauliche Arbeit ☐ oui / ja ☒ non / nein	Expert·e / Experte/Expertin (nom, prénom, E-Mail / Name, Vorname, E-Mail) Francelet Samy	

Titre / Titel

Système de gestion de bornes de recharge pour véhicules électriques**Description**

Le campus Energypolis dispose d'une flotte de bornes de recharge pour véhicules électriques dont la gestion et la supervision doivent être modernisées. L'objectif de ce projet est de concevoir et de mettre en œuvre une solution de gestion centralisée basée sur le protocole standard ouvert OCPP (Open Charge Point Protocol).

Le travail consistera à analyser l'infrastructure existante (avec un potentiel rétrofit), à sélectionner ou développer un backend de gestion (CSMS - Charging Station Management System) et à créer un Proof Of Concept qui doit permettre la supervision d'une flotte de chargeurs et de ses utilisateurs. Un suivi des consommations par utilisateur en temps réel est nécessaire. Une Dashboard de supervision et de visualisation des données de consommation devra être développée. L'implémentation d'un algorithme de répartition de charge entre les bornes devra également être considéré dans l'architecture du système.

Objectifs**Objectifs principaux (Must-have)**

- Analyse et état de l'art : étude comparative des modes de charge (AC vs DC) et des protocoles de communication. Audit technique de l'ensemble des chargeurs du Campus (compatibilité, hardware, communication, réseau, etc.).
- Analyse des plateformes de gestion et de supervision pour de bornes de charges en libre-service.
- Définition d'une architecture et mise en place de l'infrastructure OCPP pour la gestion centralisée de plusieurs chargeurs. Inclus la gestion des accès, l'authentification des utilisateurs, la gestion de plusieurs clients (bornes).
- Développement d'une Dashboard de supervision avec : suivi des consommations en temps réel, rapport par utilisateurs, interface d'administration et authentification des utilisateurs (lecteur NFC sur la borne ou application).
- Rédiger un rapport complet, fournir une documentation technique et préparer une présentation de projet.

Objectifs optionnels (Nice to have)

- Implémentation d'un algorithme de répartition de charge pour optimiser la puissance électrique disponible sur le site (smart charging ou load-balancing).
- Rétrofit des anciennes bornes de recharge à l'aide d'un contrôleur intermédiaire.
- Gestion avancée de la facturation, génération de rapport, intégration de système de facturation, etc.

DélaisDébut du travail de bachelor (TB)
Start der Bachelorarbeit (TB)**18.05.2026**

Remise du rapport final / Abgabe des Schlussberichts

14.08.2026, 12:00

Expositions et Pitch / Ausstellungen und Pitch

28.08.2026 – HEI**31.08.2026 – Monthey****03.09.2026 – Visp**

Défense orale / Mündliche Verfechtung

Semaines/Wochen 35-37 (24.08-11.09.2026)

Signature ou visa / Unterschrift oder Visum	
Responsable de l'orientation / Leiter/in der Vertiefungsrichtung 	¹ Etudiant·e / Student/in 

¹ Par sa signature, l'étudiant·e s'engage à respecter strictement la directive DI.1.2.02.07 « Travail de bachelor ». Durch seine Unterschrift verpflichtet sich der/die Student/in, sich an die Richtlinie DI.1.2.02.07 „Bachelorarbeit“ zu halten.



Electric Vehicle Charging Station Management System

Graduate

Basile Gasser

Objective

Design and development of a Proof of Concept (PoC) for a centralised monitoring platform (CSMS CitrineOS) and a dashboard (React) for the fleet of charging stations on the HES-SO Valais-Wallis campus. The aim is to provide real-time monitoring, manage users and RFID badges, and implement a dynamic power allocation algorithm (Smart Charging) using Go-based microservices.

Design a PoC for a centralized supervision platform (CitrineOS) integrating a React dashboard and Go microservices for real-time tracking, RFID access management, and intelligent power allocation (Smart Charging).

Methods | Experiences | Results

Methods: The CitrineOS CSMS was deployed on an internal server and integrated with a React user interface for centralized management. To ensure security and efficiency, a Go-based microservices architecture was developed to automate RFID badge management, detect inactive charging sessions, and execute the dynamic power allocation algorithm (Smart Charging).

Experiments: To rigorously validate the system under large-scale conditions and compensate for the obsolescence of the campus infrastructure, a heterogeneous testing environment was established. This setup combined Everest simulators (OCPP 2.1) and a Zaptec Go test station (OCPP 1.6), operating in conjunction with the compatible physical stations in the parking lot.

Results: The Proof of Concept demonstrates the feasibility of an open, sustainable, and vendor-independent centralized supervision system. The conducted tests validated the reliability of Smart Charging, secure user authentication, and real-time monitoring, establishing a robust technical foundation for the future modernization of the entire campus charging network.

Bachelor's Thesis

| 2026 |

Degree Programme
Systems Engineering

Major
Infotronics

Professor
Prof. Christopher Métrailler
christopher.metrailer@hevs.ch

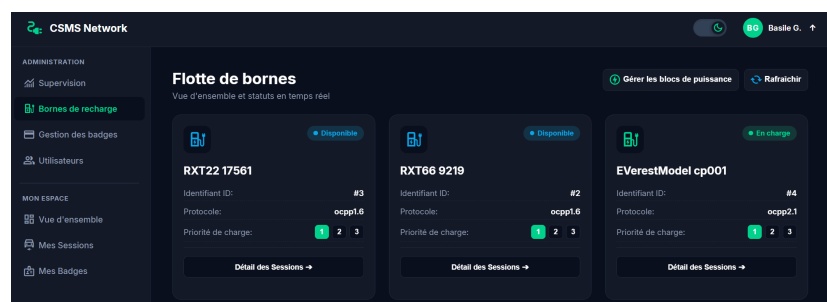


Figure 1: Electrical supply diagram of charging stations by block at the Energypolis campus.

Information about this report

Contact Information

Author: Basile Gasser
Bachelor Student
HEI-Vs

E-Mail: basile.gasser@students.hevs.ch

Declaration of honor

I, undersigned, hereby declare that the work submitted is the result of a personal work.
I certify that I have not resorted to plagiarism or other forms of fraud. All sources of
information used and the author quotes were clearly mentioned

Place, date Sion, 13.08.2026

Signature



Contents

Abstract	1
1 Introduction	2
1.1 Context and Relevance	2
1.2 Objectives and Scope (Proof of Concept)	2
1.3 Methodological Approach	3
1.4 Sustainability and SDG Alignment	4
1.4.1 SDG 7: Affordable and Clean Energy (Target 7.3 - Energy Efficiency)	4
1.4.2 SDG 9: Industry, Innovation and Infrastructure (Target 9.4 - Upgrade infrastructure)	4
1.4.3 SDG 11: Sustainable Cities and Communities (Targets 11.2 and 11.6 - Sustainable transport)	4
1.5 Thesis Structure	4
2 State of the Art and Infrastructure Audit	5
2.1 Audit of Existing Parking Infrastructure	5
2.2 Communication Protocols and Charging Modes (OCPP & AC/DC)	6
2.3 Analysis of Charging Station Management Systems (CSMS)	8
2.4 Smart Charging and Load-Balancing Concepts	9
3 Global System Architecture	10
3.1 System Overview and technology Stack	10
3.2 Requirements Specification	11
3.3 Database Design	11
3.4 Backend and Communication Architecture	12
3.5 Security and Access Design	14
3.6 Session Monitoring Logic	15
3.7 Smart Charging Logic	16
3.8 Alerting Rules and Routing	17
3.9 User Interface (UI) Design	17
3.9.1 User Interface	17
3.9.2 Administrator Interface	17
4 Backend and Microservices Implementation	19
4.1 Development Methodology & Prototyping	19
4.2 Coding Practices and Go Architecture	19
4.3 Microservices Implementation Details	20
4.3.1 Authentication and Security (auth-service)	20
4.3.2 Session Monitoring (idle-service)	20
4.3.3 Access Control (badge-service)	21
4.3.4 Smart Charging (smart-charging-service)	21
4.3.5 Alerting System (notification-service)	22
4.4 Deployment and CI/CD Pipeline	22
5 Supervision Interface (Frontend)	23
5.1 Project Organization and React Architecture	23
5.2 Network Communication and State Management	23

5.3	Design and Usability (UI/UX)	24
5.4	Main Views Development	25
5.4.1	User Interface	25
5.4.2	Administrator Interface	26
5.4.3	Detailed Views and Profile	27
6	Testing, Validation and Results	28
6.1	Test Methodology and Environments	28
6.2	Smart Charging Validation (Load-Balancing)	28
6.2.1	Scenario A: Saturation and Prioritization	28
6.2.2	Scenario B: Under-consumption and Redistribution	29
6.3	Business Flow and Frontend Validation	30
6.4	Hardware Validation	32
6.5	Results Analysis (Objectives Verification)	32
6.5.1	Validation of Main Objectives (Must-have)	32
6.5.2	Validation of Optional Objectives (Nice to have)	32
6.5.3	Additional Features	33
7	Conclusion and Future Work	34
7.1	PoC Assessment	34
7.2	Challenges Encountered	34
7.3	Future Work	34
	Glossary	36

Abstract

Background:

In 2021, HES-SO Valais-Wallis deployed a fleet of 12 charging stations on its campus. However, the obsolescence of the current infrastructure (OCPP 1.4/1.6J protocols, end of maintenance support) and the absence of a centralized Charging Station Management System (CSMS) prevent real-time monitoring, user authentication, and dynamic power management.

Objective:

This Bachelor's thesis aims to design, implement, and validate a Proof of Concept (PoC) for a centralized supervision platform based on an open-source CSMS (CitrineOS). The solution integrates a React dashboard for real-time tracking and RFID access management, as well as Go microservices ensuring security, authentication, and intelligent power allocation (Smart Charging).

Approach & Methods:

The CitrineOS CSMS was deployed on an internal server and coupled with a React user interface. To compensate for the obsolescence of the majority of the campus charging stations and to validate the system under large-scale conditions, EVerest simulators (OCPP 2.1) and a Zaptec Go test station (OCPP 1.6) were used in conjunction with the compatible physical stations in the parking lot. Dedicated microservices were developed to automate RFID badge management, the detection of inactive sessions, and dynamic charging optimization.

Results & Conclusion:

The developed PoC demonstrates the feasibility of an open, sustainable, and vendor-independent centralized supervision system. The conducted tests validate the proper functioning of Smart Charging, secure authentication, and real-time charging monitoring, thereby laying the foundation for the future modernization of the entire campus charging infrastructure.

1 | Introduction

1.1 Context and Relevance

Today, electric mobility is experiencing significant growth, and the number of electric vehicles (EVs) on the roads continues to increase. Consequently, in 2021, HES-SO Valais-Wallis undertook a transition towards electric mobility by installing a fleet of charging stations on campus. This transition aimed to reduce the carbon footprint and promote the use of electric vehicles among its staff. This fleet of charging stations was made available to personnel with permission to use the campus's private parking lot.

Currently, this charging infrastructure comprises 12 Greenmotion brand chargers, including 10 AC chargers (22kW), one DC charger (22kW), and one DC charger (66kW). Since the Swiss company Greenmotion was acquired by the Eaton group in 2021, the maintenance of these stations in Switzerland is no longer optimally guaranteed. These chargers operate with communication protocols that are now obsolete (OCPP 1.4 and 1.6J), which makes supervising these stations very difficult. Furthermore, the lack of a centralized Charging Station Management System (CSMS) prevents HES-SO from effectively monitoring the usage of these stations. As it stands, it is impossible to know which station is active, who is charging, and the total consumption in real time. Added to this is the lack of power regulation. The chargers are distributed across several power supply groups and are limited equally based on the number of chargers in a group. Therefore, even if a single charger is active, it will not be able to charge at full power and will have to limit itself to its allocated share of power. This situation is problematic as it prevents full exploitation of the infrastructure, making the user experience frustrating and inefficient.

It is now essential to implement a supervision and intelligent management solution for this fleet of charging stations. Deploying a centralized management system (CSMS) will allow for effective monitoring of the comprehensive activity of users in the parking lot. It will then be possible to identify users, track their consumption, and react more quickly in the event of charger faults. Additionally, implementing an intelligent power distribution algorithm (Smart Charging) will optimize the power available to charging vehicles based on demand and grid availability.

1.2 Objectives and Scope (Proof of Concept)

The objective of this Bachelor's thesis is to design and implement a Proof of Concept (PoC) for a centralized supervision platform, based on the OCPP protocol, enabling effective and intelligent management of the campus's charging station fleet. This platform must present a comprehensive dashboard for real-time monitoring of the stations, user management, and RFID badges for authentication at the chargers.

To achieve this, an open-source CSMS was deployed on an internal HES-SO server. The chosen CSMS is CitrineOS, notably for its compatibility with OCPP 1.6 but also 2.0.1 and higher, which will greatly facilitate the potential modernization of the fleet with more modern chargers. A React frontend will also be developed and made accessible to all users

to monitor fleet activity or simply check their own profile's current activity. To this will be added various microservices running in parallel, enabling several things:

1. Secure authentication management for the frontend.
2. Detection of inactive charging sessions (power at 0W), which indicates that a vehicle is occupying a charger needlessly.
3. Registration of unknown badges scanned at the stations to facilitate adding them in the future.
4. Smart Charging management, effectively distributing the power allocated to each station.
5. Sending notifications to concerned users or administrators.

As mentioned above, the majority of the stations operate with version 1.4 of the OCPP protocol. Given the obsolescence of this version, direct integration into the current system would require an additional adaptation layer. Since OCPP 1.4 is quite cumbersome and limited, it is preferable to interact with these stations directly via their Modbus registers. To do this, a complete mapping of the sent messages and their OCPP equivalents must be undertaken. To ensure the quality and reliability of the previously listed features as well as compliance with time constraints, this Modbus adaptation has been put on hold for future work. Given that only 2 stations operate with OCPP 1.6, in order to conduct tests on a larger fleet of stations, a total of 5 Everest station simulators operating with OCPP 2.1 were deployed on the server. This allowed for more practical and rapid experimentation with services requiring multiple active stations, such as Smart Charging. The use of OCPP version 2.1 for the simulators has effectively prepared the current system for the future of this market. In order to also study the behavior of OCPP 1.6 stations more directly, a Zaptec Go station was also set up at the workstation.

1.3 Methodological Approach

The first step in carrying out this project was the deployment of a local version of CitrineOS. This allowed for practical and rapid experimentation to effectively implement the basic functionalities built around the CSMS. A first version of the frontend was created to quickly provide a conceptual visualization of what was possible.

Once the server was made available, a second, this time remote, version of CitrineOS was set up. To keep development simplified, the frontend remained local to benefit from React's dynamic updating (Hot-Reloading) in development mode, while communicating with the CitrineOS database via an SSH tunnel. The Go services were then developed one after another while keeping the frontend consistent with the new additions.

The entire system was kept up to date using the GitHub versioning system.

After the implementation of each feature, the Everest simulators and the Zaptec charger served as a test bench, allowing almost immediate experimentation and validation of most features. The 2 OCPP 1.6 chargers in the parking lot were also quickly commissioned and linked to the CSMS to conduct tests in real-world conditions.

1.4 Sustainability and SDG Alignment

This charging infrastructure modernization project not only responds to a technical need but actively contributes to HES-SO's sustainability approach by directly addressing three United Nations Sustainable Development Goals (SDGs).

1.4.1 SDG 7: Affordable and Clean Energy (Target 7.3 - Energy Efficiency)

The Smart Charging service enables effective energy optimization. Rather than statically limiting the power of each station, the system allocates maximum capacity to charging vehicles based on the number of active stations. It makes the use of the station fleet more dynamic while guaranteeing compliance with the power limits of each power group.

1.4.2 SDG 9: Industry, Innovation and Infrastructure (Target 9.4 - Upgrade infrastructure)

The adoption of the open standard OCPP (Open Charge Point Protocol) and the deployment of an Open Source CSMS allows the system to completely free itself from any dependence on a proprietary manufacturer. This open architecture guarantees the longevity of the installation, facilitates the integration of new stations (AC or DC), and ensures high adaptability to future technological developments.

1.4.3 SDG 11: Sustainable Cities and Communities (Targets 11.2 and 11.6 - Sustainable transport)

By offering centralized supervision and real-time monitoring of station availability, the platform makes the charging service more reliable for campus staff. This helps remove obstacles related to the daily use of electric vehicles, thereby encouraging the transition towards low-carbon mobility within HES-SO Valais-Wallis.

1.5 Thesis Structure

The remainder of this document is structured as follows:

- State of the Art and Infrastructure Audit: Presents a comprehensive technical analysis of the technologies, protocols, and standards used throughout the project.
- Global System Architecture: Details the overall structure of the hardware and software infrastructure as a whole.
- Backend and Microservices Implementation: Explains in detail the implementation and configuration of CitrineOS as well as the Go microservices.
- Supervision Interface (Frontend): Covers the organization, implementation, and deployment of the user interface developed in React.
- Testing, Validation and Results: Presents a report on the testing and validation of all system components.
- Conclusion and Future Work: Synthesizes the results obtained and presents evolutionary perspectives for the future of this infrastructure.

2 | State of the Art and Infrastructure Audit

2.1 Audit of Existing Parking Infrastructure

The parking infrastructure currently consists of 12 GreenMotion chargers of 3 different types: 10 AC Private One 22 chargers [1], 1 DC XT22 charger [2], and 1 DC XT66 charger [3].

Charger	Type	Connectors	OCPP Version	Simultaneous charging
Private One 22	AC	Type 1 \ Type 2	1.4	No
Range XT22	DC	CCS2 \ CHAdeMo	1.6J	No
Range XT66	DC	CCS2 \ CHAdeMo	1.6J	No

Table 1: Comparison of charger models.

All charging stations are distributed across several different power supply groups.

- **Block 1** : 400V/125A (86,6kW) -> 1 x XT66
- **Block 2** : 400V/63A (43,6kW) -> 4 x Private One 22
- **Block 3** : 400V/32A (22.2kW) -> 4 x Private One 22
- **Block 4** : 400V/63A (43,6kW) -> 2 x Private One 22 and 1 x XT22

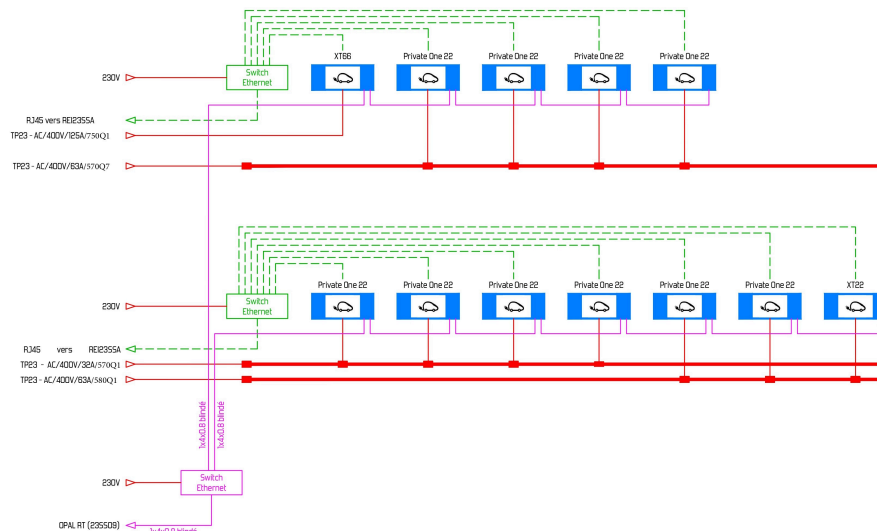


Figure 2: Electrical supply diagram of charging stations by block at the Energypolis campus.

As shown by the distribution of charging stations across the different power supply groups, only one block provides enough energy to fully satisfy the demand. With the current static limitation system, some chargers are throttled to only a quarter of their maximum power. This further proves the necessity of implementing a Load Balancing/ Smart Charging algorithm.

Eaton's acquisition of Greenmotion in 2021 raises several other issues. Eaton's commercial strategy is centered on their global ecosystem, imposing the use of paid licensed software. Furthermore, older charging stations like the Private One see their firmware updates reduced, favoring the use of newer models. As Greenmotion was previously a Swiss-based company, support was much more flexible and accessible. With the transition under Eaton's international corporate structure, this technical support becomes significantly less adapted and accessible.

2.2 Communication Protocols and Charging Modes (OCPP & AC/DC)

Communication between electric vehicles and charging stations is governed by several different standards and also depends on the type of charging (AC or DC) used.

AC Charging

AC charging communication is governed by the IEC 61851-1 standard [4]. This standard stipulates that communication between the vehicle and the charging station must occur via a 1kHz PWM signal emitted on the connector pin named "Control Pilot" or CP. Modulating the Duty Cycle and amplitude of this signal allows defining several charging modes and states.

Amplitude Modulation: The amplitude of the CP signal defines the connectivity state between the vehicle and the charging station.

- 12V : the vehicle is not connected to the charging station
- 9V : the vehicle is connected to the charging station but does not request a charge
- 6V : the vehicle is connected to the charging station and requests a charge
- 3V : the vehicle is connected to the charging station, requests a charge but requires external ventilation (very rare on modern vehicles)
- 0V : The CP pin is short-circuited, which prevents charging and indicates a fault
- -12V : The station is unavailable following a restart or a fault.

Duty Cycle Modulation: A very low or very high DC indicates a specific fixed state, while a DC ranging from 10% to 85% indicates a linear amperage according to the following ratio: $I = DC * 0.6$.

- DC = 0% : The station reports an error or is out of service
- DC < 3% : The station refuses the charge
- $3\% \leq DC < 7\%$: Switches to digital communication mode (ISO 15118 / DIN 70121)
- $7\% \leq DC < 10\%$: Unused range reserved for future use
- $10\% \leq DC < 85\%$: Linear amperage according to the ratio $I = DC * 0.6$
- $85\% \leq DC < 96\%$: Specific amperage according to the ratio $I = (DC - 64) * 2.5$
- $96\% \leq DC < 100\%$: The station reports an error or is out of service
- DC = 100% : Station on standby, no vehicle connected

DC Charging

Regarding DC charging communication protocols in Europe and America, two standards must be considered: ISO 15118 [5] and DIN 70121 [6]. These two standards define communication protocols for DC fast charging and allow bidirectional communication between the vehicle and the charging station. The DIN 70121 standard, being somewhat the ancestor of the ISO 15118 standard, is increasingly less used and tends to be replaced by the ISO 15118 standard. To enable communication between the car and the charging station, ISO 15118 relies on communication based on the OSI model. The vehicle and the station exchange data packets using standard web protocols: IPv6 and TCP.

However, in Japan and China, two other standards are used: CHAdeMO and GB/T. These standards are specific to these regions and are not compatible with the ISO 15118 and DIN 70121 standards used in Europe and America. Communication is based on a dedicated CAN bus.

OCPP

OCPP (Open Charge Point Protocol) is the standard open reference protocol ensuring cooperation between the charge point and the Central Charge Point Management System (CSMS). By decoupling the physical infrastructure from the management software layer, OCPP implementation prevents any risk of vendor lock-in. This technical neutrality allows integrating equipment from different manufacturers into a single network, thereby ensuring the longevity and scalability of charging station fleets in the face of future industry developments.

The OCPP protocol architecture has undergone significant structural modifications across its revisions. Version OCPP 1.4, historically based on the SOAP/XML protocol, now presents significant technical obsolescence, notably transmission rigidity, justifying the impossibility of its direct integration without an adaptation gateway. The emergence of OCPP 1.6J [7] marked a decisive transition thanks to the adoption of the JSON format routed via WebSockets. This version introduced low-latency bidirectional communication as well as the first native smart charging features.

Henceforth, the OCPP 2.0.1 and 2.1 [8] specifications represent the ideal technological target for modern networks. They integrate an enhanced security framework, a fully modular data model by components, and native support for advanced features such as the ISO 15118 standard (Plug & Charge, V2G) or the EVerest open-source ecosystem. The table below summarizes these evolutions and specifies the implementation status within the PoC developed in this work.

Version	Transport / Format	Security	Smart Charging Support	Status in the PoC
OCPP 1.4	HTTP \ SOAP (XML)	Low (no native encryption)	Limited	Not supported
OCPP 1.6J	WebSockets \ JSON	Medium (optional TLS)	Basic profile (Smart Charging Profile)	Physical DC stations
OCPP 2.0.1, 2.1	WebSockets \ JSON	High (mandatory TLS, PKI)	Advanced (ISO 15118, V2G, EVerest)	EVerest Simulators

Table 2: Comparison of OCPP versions.

2.3 Analysis of Charging Station Management Systems (CSMS)

The Charging Station Management System (CSMS) constitutes the central component of this project, around which the entire software architecture revolves. Its role is essential: maintaining real-time communication with the stations, driving transactions, and guaranteeing data storage reliability. To choose the most suitable solution, the following criteria must be considered:

- support for OCPP versions,
- database modularity and scalability,
- the ability to integrate easily into the current ecosystem,
- the overall quality of the project's documentation.

SteVe [9]

- OCPP Support: Made for OCPP 1.6J but completely obsolete compared to newer versions.
- DB Modularity and Extensibility: Fixed and aging architecture (Java/Spring) and rigid data schema.
- Ease of Integration: Easy to deploy on a server via Docker.
- Vitality & Documentation: Very well-documented project but no updates for some time (obsolete).

MaEVe [10]

- OCPP Support: Designed for OCPP 2.0.1 but partially compatible with OCPP 1.6J.
- DB Modularity and Extensibility: Adding features or modifying current structures requires rewriting part of the microservices, which could easily create conflicts with the rest of the system.
- Ease of Integration: Made for deployment on a public cloud with a Kubernetes orchestrator. Designed for larger-scale projects (thousands of stations).
- Vitality & Documentation: Very active development and regularly updated.

CitrineOS [11]

- OCPP Support: Natively handles recent versions like OCPP 2.0.1/2.1 and also OCPP 1.6J via dedicated conversion modules.
- DB Modularity and Extensibility: PostgreSQL database structured in separate modules. Allows easy additions of SQL tables/columns without impacting the rest of the structure. Simplified usage via the Hasura graphical interface.
- Ease of Integration: Easy to deploy on a server via Docker.
- Vitality & Documentation: Active project in constant development. Comprehensive and reliable documentation.

For this project, CitrineOS emerged as the most suitable CSMS. It allows for simplified implementation with 1.6J stations and keeps the architecture ready for future developments. The modular structure of the PostgreSQL DB enabled rapid and simplified implementation of new features.

2.4 Smart Charging and Load-Balancing Concepts

In an installation like the HES-SO Energypolis Campus, it is essential to comply with certain electrical consumption rules. As mentioned above, the charging stations are distributed among several power supply groups with a fixed consumption limit. Exceeding this limit could trigger the circuit breakers, thereby cutting off all active charging on the group. Currently, each charger is allocated a fixed share of the available power that it must never exceed. So, if only one charger is active, it will still be throttled to the previously defined power share. A more effective approach would be to dynamically define the limits based on the number of active stations within the same block.

Static allocation: If on a 22kW power block there are 3 chargers but only one is active, it will only have 7.3 kW.

Dynamic allocation: if on a 22kW power block there are 3 chargers but only one is active, it will have the full 22kW available. When another charging session starts, the limits will then be redefined to allocate 11kW to each session.

However, it happens that a vehicle consumes less than the limit applied to the station, particularly if the vehicle is less performant than the station or if it is nearing the end of its charge. In a context of shared power among multiple vehicles, this can lead to a scenario where a station “wastes” energy, unnecessarily slowing down the charging of other vehicles. This is where the concept of “Smart Charging” comes into play. An algorithm will be responsible for detecting the under-consumption of the stations by comparing the consumption measures to the applied limit, in order to recalculate the limits in real time.

To communicate its limits to the stations, OCPP provides a series of possible instructions via the “SetChargingProfile”. It is possible to send two types of charging profiles to the stations: the default charging profile, applied at the beginning of each session, and the direct charging profile, which dynamically changes the consumption limit for an active session. The details of the sending methods as well as the format will be detailed in Section 4.3.4.

3 | Global System Architecture

3.1 System Overview and technology Stack

Deployed on a Linux server, CitrineOS is provided with a PostgreSQL database and an intermediary API engine, Hasura, allowing data to be distributed to the frontend. The microservices run in parallel and receive the desired information from the database via a series of Event Triggers, configured from the Hasura web interface, and write to the tables via SQL commands. The Smart Charging service also communicates the charging profiles via the CitrineOS API. The five EVerest simulators communicate with CitrineOS in OCPP via an internal Docker network. The Zaptec station is connected via WebSocket through port 8081, while the GreenMotion stations communicate via a local subnet (LAN) of the HES configured by the IT department.

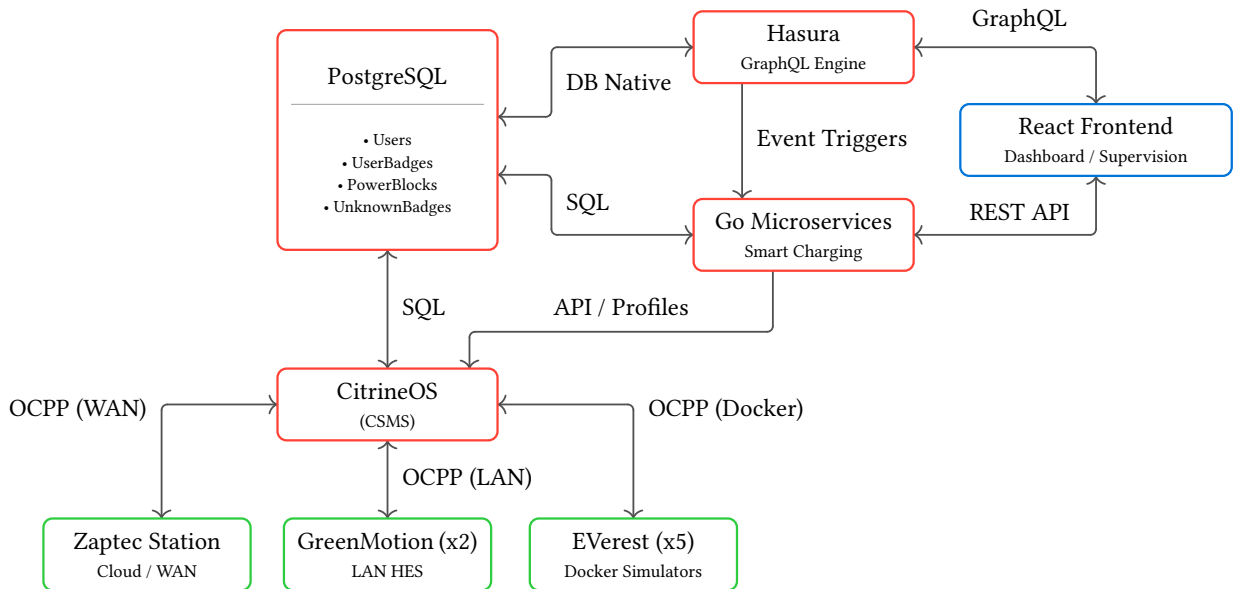


Figure 3: CSMS System Architecture - Energypolis Campus

The language chosen for the development of the external microservices is Go (Golang). This compiled language offers extremely fast execution times and very low memory consumption (RAM/CPU) compared to environments like Node.js, Python, or Java. Furthermore, native concurrency management allows the microservices to easily process a large number of events and requests simultaneously.

The frontend, on the other hand, was developed with the React library. It is the most popular industry standard for web interface development. Due to this predominance, large language models (LLMs) have been trained on a massive volume of high-quality React code. This synergy with artificial intelligence significantly accelerates interface prototyping and effectively integrates most features (A detailed description of the Artificial Intelligence tools used and their framework of use is available in Appendix AE).

3.2 Requirements Specification

This work is divided into two main objectives. First, the deployment of CitrineOS and its surrounding ecosystem (Hasura/PostgreSQL). The system must detect connected charging stations, handle charger control, and ensure the management of charging sessions and access.

Second, the development of a supervision Dashboard. This platform must allow for real-time monitoring of consumption and the general state of the station fleet. It must also manage user access to the stations, as well as global administrator supervision of the system (management of chargers, users, and NFC/RFID badges).

Among the secondary objectives, the Smart Charging and load-balancing algorithm was selected and fully implemented. Several additional features (spontaneous or required during development) were also added, such as user profile management, unknown badge management, the notification system, inactive session detection, and the detailed configuration of power supply groups.

3.3 Database Design

The structure of CitrineOS's native PostgreSQL database had to be extended several times to effectively meet the specific needs of this work. A diagram detailing the modified part of the database structure is presented in Appendix AC.

By default, the database manages NFC badges (via the Authorization table) but not their ownership by a specific user. The first extension was therefore the complete storage of users (personal information and login credentials) as well as their link to the badges. To facilitate implementation and modularity, a Users table was created, then linked to a UserBadges join table. This architecture allows a user to own multiple badges, while ensuring that a badge can only belong to one user at a time.

In the initial structure, the only linking information contained in the Transactions table (representing charging sessions) is the badge used. Since a badge can now change owner, an explicit link had to be added between the charging session and the badge owner at the time the session was initiated.

The management of power supply groups also required an extension via the addition of the PowerBlocks table. This table contains the electrical characteristics of a block (maximum current, voltage, maximum power). To link these blocks to the different stations, a power_block_id column was added to the ChargingStations table. It is to these data that the Smart Charging algorithm refers for the dynamic adjustment of consumption limits. To ensure the reliability of controlling these limits, an allocated_limit column was also added to the Transactions table.

The storage of unrecognized badges required the addition of an UnknownBadges table. This avoids having to parse the entire history of OCPP communications to find these events. This table only contains the NFC identifier of the badge, the timestamp of its detection by the station, and the number of scan attempts.

Finally, the Transactions table received a final adjustment: the `is_legal` (validity status) and `overtime_start_timestamp` columns were implemented to mark a charging session as inactive and to record the exact moment when the transaction stopped consuming energy.

3.4 Backend and Communication Architecture

For the microservices to detect changes in the database, the first solution considered would have been polling. This solution could have worked but would have generated far too many unnecessary requests. To overcome this, an event-driven architecture based on Event Triggers was configured in Hasura.

An Event Trigger automatically detects a change in a database element (entire table or specific column) and generates HTTP POST requests (Webhooks) targeting the internal ports of the microservices. This ensures that business logic is executed only when data evolves in the database, thereby optimizing the use of system resources.

Here is the configuration of the event triggers for each domain of the system:

For Session Monitoring (idle-service):

- `track_transactions` : Transactions (INSERT, UPDATE) targeting http://tb_csms-idle-service:8080/webhook
- `track_metervalues` : MeterValues (INSERT) targeting http://tb_csms-idle-service:8080/webhook

For Access Control (badge-service):

- `track_ocpp_messages` : OCPPMessages (INSERT) targeting <http://badge-service:8082/webhooks/messages>

For Dynamic Charge Management (smart-charging-service):

- `smart_charging_transactions` : Transactions (INSERT, UPDATE) on the `isActive` column targeting <http://smart-charging-service:8081/webhooks/transactions>
- `smart_charging_power_blocks` : PowerBlocks (UPDATE) on the `max_a`, `n_phase`, `max_v` columns targeting <http://smart-charging-service:8081/webhooks/transactions>
- `evse_assignment_trigger` : ChargingStations (INSERT, UPDATE) on the `power_block_id`, `protocol` columns targeting <http://smart-charging-service:8081/webhooks/station-assignment>
- `smart_charging_station_reconnection` : ChargingStations (UPDATE) on the `isOnline` column targeting <http://smart-charging-service:8081/webhooks/transactions>
- `station_weight_trigger` : ChargingStations (INSERT, UPDATE) on the `weight`, `power_block_id` columns targeting <http://smart-charging-service:8081/webhooks/transactions>

For the Alert System (notification-service):

- `notify_password_reset` : Users (UPDATE) on the `reset_token` column targeting <http://notification-service:8084/webhooks/notify>
- `notify_unknown_badge` : UnknownBadges (INSERT) targeting <http://notification-service:8084/webhooks/notify>

- notify_idle_transaction : Transactions (UPDATE) on the is_legal column targeting <http://notification-service:8084/webhooks/notify>
- notify_wait_for_energy : Transactions (UPDATE) on the allocated_limit column targeting <http://notification-service:8084/webhooks/notify>
- notify_connector_error : Connectors (UPDATE) on the errorCode column targeting <http://notification-service:8084/webhooks/notify>

3.5 Security and Access Design

Given that the architecture relies on Hasura as the primary engine for exposing the database, security must be managed decentrally. Hasura entirely delegates identity verification to an external component that acts as an authentication provider.

The chosen integration design relies on issuing a JWT token. This token is designed to include a JSON object into which the context variables essential for Hasura are injected, namely `x-hasura-user-id`, `x-hasura-default-role`, and `x-hasura-allowed-roles`. It is the presence of these variables, sealed in the token, that allows Hasura to automatically apply the security rules configured on the various database tables during GraphQL requests issued from the frontend.

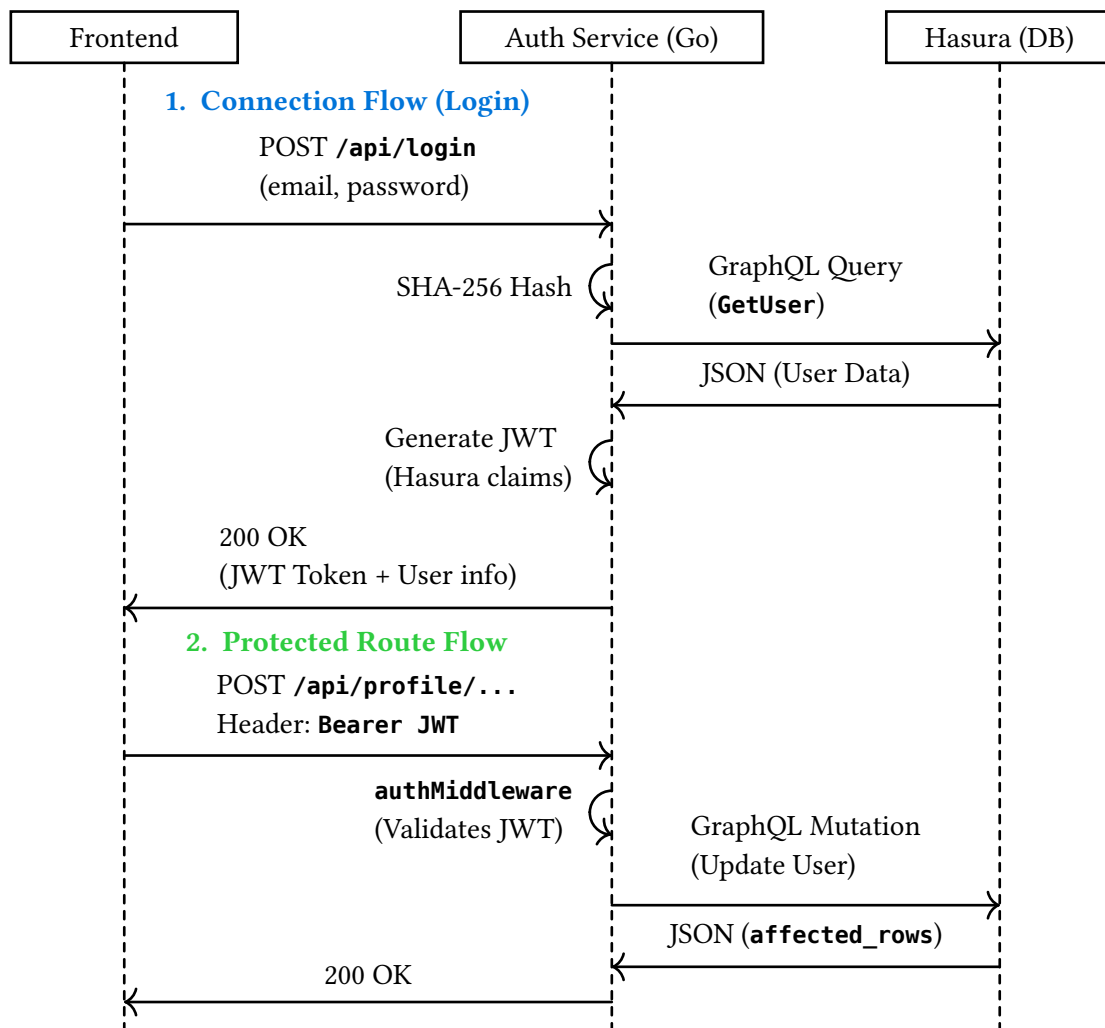


Figure 4: Sequence diagram for authentication and authorization.

3.6 Session Monitoring Logic

The monitoring of charging session activity relies on a state machine modeling, distinguishing legal sessions from violating sessions. A transaction transitions from a legal state to a violation state when the vehicle occupies the spot without consuming energy beyond an authorized grace period. Conversely, the effective resumption of energy transfer resets the monitoring and maintains or restores the compliant status of the session.

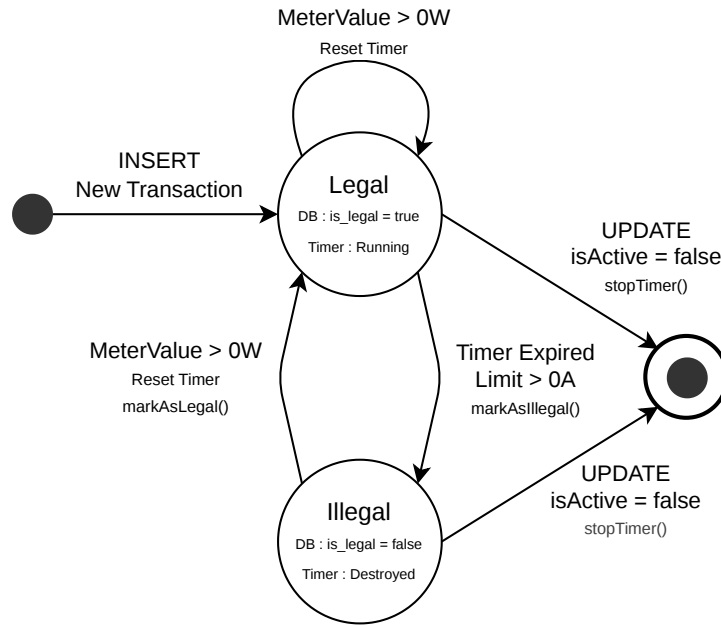


Figure 5: State diagram for transaction management.

The raw algorithmic logic to process this transition relies on querying the allocated limit and managing a timer whose expiration validates or invalidates the violation.

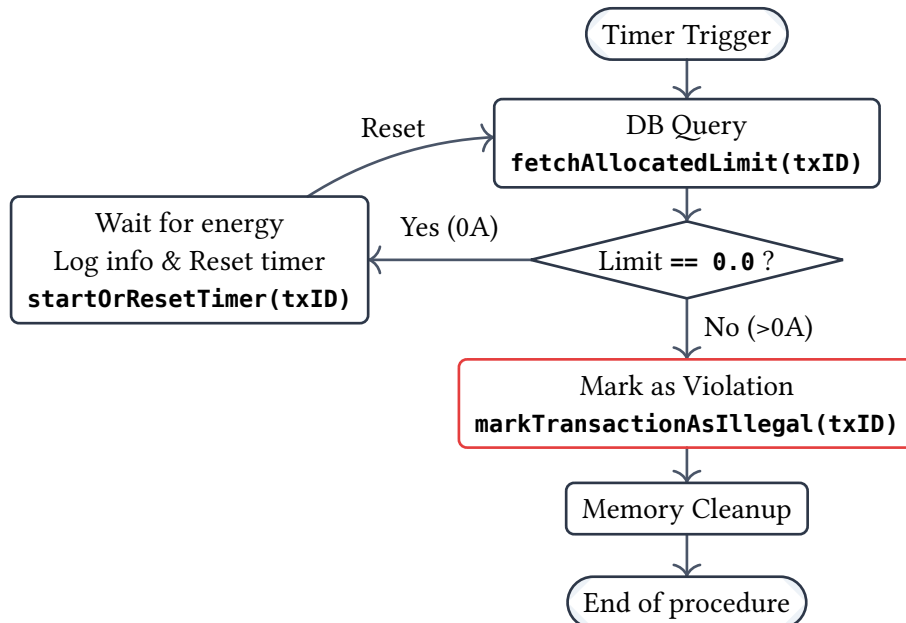


Figure 6: Flowchart for inactivity verification.

3.7 Smart Charging Logic

The Smart Charging algorithm is divided into four distinct steps:

1. The first step determines whether a transaction can start consuming or not. The minimum current a charger must provide is set to 6A. If the available current in the relevant power block does not allow providing this minimum to all sessions, those with the lowest priority or the most recent arrival time are suspended.
2. Once a transaction is launched, a periodic verification of its consumption begins. The goal is to detect if a vehicle is consuming less than its allocated current (underloading phenomenon). If this is the case, the limit is redefined just above the vehicle's actual consumption.
3. After allocating the vital minimums or following the detection of an under-consuming vehicle, the remaining available current in the block is redistributed to active transactions based on the priority level of each station. Any remaining amperes, due to rounding during calculations, are then allocated to the highest priority stations.
4. During the testing phase with the physical OCPP 1.6J stations, the latter proved to be very intolerant of excessively frequent limit changes. Upon receiving a new instruction, these stations go through a transition phase where the transaction is suspended for approximately one minute. Sending a new instruction during this phase generally causes a hardware error. To overcome this problem, the limit of the OCPP 1.6J stations is only modified if the variation is at least 3A. Furthermore, a 5-minute cooldown period was imposed between each state change to prevent the station from constantly being in transition (with the exception of emergency cut-offs at 0A).

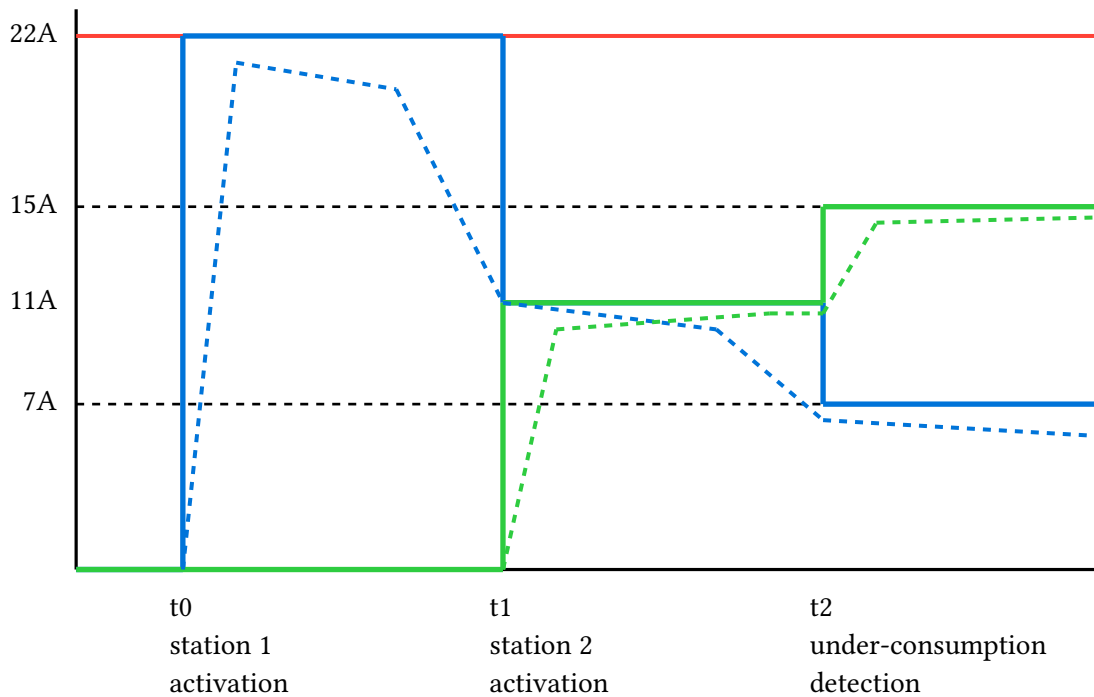


Figure 7: Chronogram illustrating the temporal evolution of current allocation by Smart Charging.

3.8 Alerting Rules and Routing

The alerting system defines strict routing rules to target the right recipients based on internal system trigger conditions:

- End of charge (Violation): Triggers when an ongoing transaction becomes illegal (inactivity). An email is sent to the targeted user asking them to move their vehicle which is unnecessarily occupying the space.
- Waiting for or resuming energy: Activates if the allocated current limit drops to zero or goes back above zero. The user owning the session is notified of the temporary interruption or the automatic resumption of their charge.
- Unknown badge: Triggered by the detection of a denied RFID connection attempt. An alert is routed to all system administrators to report the event, facilitating manual addition from the dashboard if the visitor is legitimate.
- Hardware error: Occurs when the operational state of a physical connector differs from its nominal state. Administrators are immediately alerted of the physical fault in order to plan a maintenance intervention.
- Password reset: Follows the generation of a security token on a profile. The concerned user receives a unique and temporary link allowing them to configure a new password.

Examples of Emails can be found in the Appendix AF.

3.9 User Interface (UI) Design

3.9.1 User Interface

A standard user has access to three distinct tabs:

- Overview: This tab indicates the number of charging stations currently available in the parking lot. It also displays the user's active charging session(s) in real time (or the last session completed). General statistics are added, such as total consumption and a weekly breakdown over the last 16 weeks.
- My Sessions: This page provides access to the complete history of charging sessions performed by the user.
- My Badges: This section lists the RFID/NFC badges linked to the profile. The user has the ability to add new badges or modify existing ones.

3.9.2 Administrator Interface

For users with administrative permissions, four additional tabs are added to the standard interface:

- Supervision: This space allows for complete monitoring of the fleet. It displays the status of the chargers (active, available, offline), the instantaneous power consumed by the system, and the total energy distributed during the day. A graph traces the evolution of consumption over the last 24 hours, accompanied by the list of ongoing sessions.
- Charging Stations: This page lists all registered hardware. Here, it is possible to adjust the priority level of each charger, view the history of their sessions, and access the configuration of power blocks. Administrators can thus create, modify, or delete power supply groups and assign the corresponding stations to them.

- **Badge Management:** Similar to the user tab, this view allows the global management of all badges registered in the system as well as access to the session history filtered by badge.
- **Users:** This tab centralizes profile management. The administrator can modify personal information, adjust permission levels, and view a summary of transactions per user.

From any of these tabs, clicking on a transaction allows viewing its precise details. This view includes key figures (total consumption, duration) as well as the charging power variation curve. Administrators also have an activity log listing the technical events that occurred during the session (a feature exclusive to sessions operating under OCPP 2.0.1 and higher).

Finally, a tab dedicated to profile configuration allows any user to update their personal information, email address, and password.

4 | Backend and Microservices Implementation

4.1 Development Methodology & Prototyping

The first step that had to be validated was the setup of CitrineOS. Once the minimal environment was deployed, along with an EVerest simulator to verify proper operation, an initial version of the Dashboard was developed. This version aimed to implement a preliminary user management system and an initial supervision interface including the management of charging stations and badges present in the system.

In a second phase, a Linux server was provisioned to deploy the system. Alongside this, the two DC chargers in the parking lot were also configured to communicate with this server. Once CitrineOS was deployed on it, a second version of the Dashboard was created, adding the details of charging sessions such as duration and consumption curves, and integrating a differentiation between user and administrator roles.

As the project progressed and adjustments were made, the microservices were implemented and tested with actual charging sessions on the physical stations where possible, while keeping the Dashboard updated to remain consistent with new features (such as the visualization of the inactive status).

During this development phase, the test environment featuring the five EVerest simulators enabled rapid and optimal work. Without direct access to the parking lot with a vehicle, any test requiring interaction with one or multiple stations (such as scanning an unknown badge or load balancing) could not have been conducted smoothly.

4.2 Coding Practices and Go Architecture

To make the code lightweight and performant, Go's standard net/http library was used for the entire networking foundation. It allows for the straightforward creation of endpoints for the microservices to receive incoming requests, as well as to issue HTTP requests to other systems, such as GraphQL or REST APIs.

Through net/http, the Go HTTP server automatically launches a distinct goroutine for each incoming request. If a request triggers a lengthy process involving complex calculations or external API calls, keeping the HTTP connection open can lead to timeout errors. To mitigate this phenomenon, the endpoints simply receive the request data, launch the actual processing in a new dedicated goroutine, and then immediately return a 200 OK success code. This ensures that the microservices never block incoming network flows.

Since multiple goroutines run in parallel, they occasionally attempt to access the same memory variables or structures simultaneously. To prevent race conditions, the critical resources in the code are locked using sync.Mutex synchronization primitives. This forces the goroutines to wait until the memory area is released by the occupying goroutine before they can read or write data to it.

In the event of a request interruption or cancellation, Go goroutines lack an implicit native shutdown mechanism. The architecture therefore heavily leverages the context package. The latter is passed as a parameter throughout the entire call chain to propagate cancellation signals and manage timeouts. If a context is canceled, all goroutines associated with this context intercept the signal and terminate cleanly, thereby freeing up the CPU and memory.

To ensure the application's robustness when dealing with remote data, strict typing management was implemented via the definition of specific structs. To secure the decoding of JSON streams, custom serialization overrides were programmed. Unpredictable dynamic data is received via empty interfaces (`interface{}`) and then carefully manipulated using safe type assertions, thus avoiding any critical runtime errors (panics). Finally, the architecture maintains a strict separation of concerns: the code is divided into distinct modules separating HTTP handlers, the database communication layer, and software clients.

4.3 Microservices Implementation Details

4.3.1 Authentication and Security (auth-service)

The implementation of the authentication service is based on the generation and validation of tokens according to industry standards. The code was structured around the standard `golang/jwt/v5` library. When a login request provides valid credentials, the service constructs a JWT token by injecting user data into it, and cryptographically signs it using the HMAC-SHA256 algorithm paired with a secret key (`JWT_SECRET`) loaded from environment variables.

To protect profile modification operations within the REST API, a middleware named `authMiddleware` was coded. This component intercepts all requests directed towards the service's private routes before they reach their destination logic. Its implementation consists of extracting the value of the HTTP Authorization header, then syntactically verifying the presence of the Bearer prefix. The extracted token is then parsed by the `Parse` function of the JWT library, which mathematically validates the signature via the same secret key, while ensuring that the expiration property has not lapsed. If the token fails any of these checks or is missing, the middleware interrupts the processing chain and instantly returns an HTTP 401 Unauthorized response.

4.3.2 Session Monitoring (idle-service)

The implementation of the inactivity detection service relies on passive HTTP event handlers. The code executes only when the service receives a request on its endpoints configured to listen to the database.

The temporal monitoring logic was implemented using the `time.AfterFunc` function from Go's standard library. When a transaction starts or an active power measurement is reported, the service instantiates an asynchronous timer set to a duration defined by the `GRACE_PERIOD_MINUTES` environment variable. This function returns a pointer to a `Timer` object. To track these objects in memory, the service uses a map data structure associating the transaction identifier with its current timer, all protected by a `Mutex`.

If new consumption measurements are received for a vehicle, the service searches for the corresponding timer in the shared memory. The `Stop()` method is called on the instance to cancel its execution, then a new `time.AfterFunc` timer is recreated and assigned to the map. Finally, when an event signals the definitive end of the transaction, the timer is stopped if it was active, and the corresponding entry is permanently deleted from memory via the internal `delete()` function. This rigorous manual reference management guarantees the absence of memory leaks during the continuous operation of the application.

4.3.3 Access Control (badge-service)

The software implementation of the badge management service aims to intercept authorization denials. The main technical challenge lay in the fact that the error payloads transmitted by the system did not contain the character string (UID) of the offending badge.

To circumvent this lack of information, the code was structured to exploit the correlation identifier (`correlationId`) present in the error request. Upon receiving the failure message, the service compiles and executes a dynamic GraphQL query targeted at the communication history. This query uses the `correlationId` as a filter to retrieve the initial authorization request message. The service then parses the JSON response to extract the raw UID of the badge. The code then verifies its existence within the database structures via new queries, and performs a final insertion mutation if the badge must be registered as unknown.

4.3.4 Smart Charging (smart-charging-service)

The implementation of the dynamic load management algorithm is based on the dynamic creation of lightweight processes. When a software update concerning a power block is detected via the HTTP entry points, an autonomous goroutine is instantiated specifically for the concerned block. This routine integrates an infinite calculation loop, clocked by Go's `time.Ticker` object.

To avoid the accumulation of these infinite loops and the ensuing memory leaks, the life-cycle of each goroutine is strictly controlled via the context package. The implementation retains the cancellation function (`CancelFunc`) of each process in memory. Thus, if a new HTTP request signals a state change on a block already undergoing calculation, the code explicitly calls the cancellation function of the old loop before instantiating a new one with updated parameters.

Within this evaluation loop, the code must impose a minimum delay between setpoint changes directed at the physical stations. The use of the blocking `time.Sleep()` function was prohibited, as it would pause the entire calculation process for the block. Instead, an in-memory caching system records the timestamp of the last transmission for each station. At each iteration, the `time.Since()` condition evaluates the elapsed time against this cache. If the minimum delay is not met, the `continue` keyword is executed, allowing the program to ignore the relevant station while immediately continuing the evaluation of other transactions within the same block.

Finally, the effective application of the calculated mathematical limits requires sending network requests (HTTP POST) to the chargers. To avoid penalizing the execution time of the main loop, the `updateTransactionLimitInDB` function is called asynchronously

using the `go` keyword. This approach delegates the processing of network I/O to parallel ephemeral subroutines, thereby maximizing the performance of the calculation engine. The detailed Smart Charging algorithm is available in Appendix AD.

4.3.5 Alerting System (notification-service)

The implementation of the alerting service relies on processing the various data received on the endpoints, and the technical dispatch of emails. The sending of electronic messages relies entirely on Go's standard `net/smtp` library.

When a logical process is triggered by an incoming HTTP request, the Go code formulates an event-context-specific GraphQL query to traverse the table hierarchy and extract target email addresses. The SMTP module then formats the header, body, and metadata of the email, and opens a network connection to the mail server configured by the school. This modular implementation choice using the standard SMTP port allowed the component to adapt without any software modification to the network constraints of the Linux server, making the microservice operational as soon as the infrastructure's network rights were granted.

4.4 Deployment and CI/CD Pipeline

The continuous integration of the project relies on the use of GitHub Actions, ensuring smooth automation of the development cycle. A Git workflow was specifically configured to react to every new modification pushed to the code repository. To optimize processing times and resource usage on the integration servers, the system is capable of precisely detecting which source code directories have been modified. Since the overall architecture is split into several distinct modules, only the Docker images corresponding to the altered components are rebuilt by the automation scripts. Once compiled and generated, these new images are automatically pushed and stored in the container registry, ready for deployment.

Although the executable build phase is fully automated, the actual deployment to the production Linux server was deliberately kept as a manual process. This implementation approach maintains strict engineering control over the production release. In practice, the update is performed in a command-line environment on the target server. Executing the `docker compose pull` command queries the registry to retrieve the latest versions of the containerized binaries, and then `docker compose up` forces the runtime engine to recreate and restart the relevant services. Detailed instructions for updating the system components are documented in Appendix AA.

5 | Supervision Interface (Frontend)

5.1 Project Organization and React Architecture

The frontend project is developed using React and TypeScript, which ensures strong static typing for the entire codebase and guarantees greater robustness. The file structure has been designed to clearly separate responsibilities, following a modular architecture.

Within the main src directory, the code is organized into several key folders:

api : centralizes all communication functions with the backend and the database (authentication, charging stations, sessions, etc.).

components : groups the reusable visual elements of the interface, with a layout subfolder dedicated to the overall structure of the application.

context : contains the global state providers, used for authentication management and theme application.

pages : segments the different views of the application, with a clear separation between the authentication screens and the dashboards (divided between administrator and user).

Files located at the project root, such as types.ts or theme.css, complete this structure to define the global TypeScript interfaces and the application's graphic charter, respectively.

5.2 Network Communication and State Management

All network communications in this project rely on the native JavaScript fetch API. To standardize data exchanges with the database, a custom utility function (fetchHasura) was developed. It is responsible for encapsulating GraphQL requests as standard HTTP POST requests to the Hasura engine, while properly handling returned errors to transmit them to the user interface. This same native approach is used to communicate with the Go-developed microservices.

In the context of this Proof of Concept (PoC), data retrieval occurs upon the initial loading of the components. Data updating is done transactionally or via a manual refresh by the user, which significantly reduces the load on the central server.

State management fully leverages native React tools. The application's global state, including connection status and profile information, is centralized using the Context API. Local states, such as form data or lists, are managed in isolation within each component via the useState hook.

Regarding security, the password is hashed in SHA-256 on the client side before being transmitted during account creation. Subsequently, the JWT token generated by the authentication service is stored in the browser (localStorage). The request function automatically intercepts this token to inject it into the HTTP header of every outgoing request. A global validation logic allows for a forced logout to be triggered if the token is expired or deemed invalid.

5.3 Design and Usability (UI/UX)

The visual aspect of the platform was entirely custom-designed. The interface relies exclusively on pure CSS combined with inline styles typed via React.CSSProperties, which prevents syntax errors while maintaining a dynamic approach.

The application is structured around a global style file defining a set of CSS variables. This approach enables centralized management of the graphic charter and facilitated the native implementation of a light mode and a dark mode. Navigation revolves around a main layout component that encompasses a top navigation bar (Topbar) and a side menu (Sidebar).

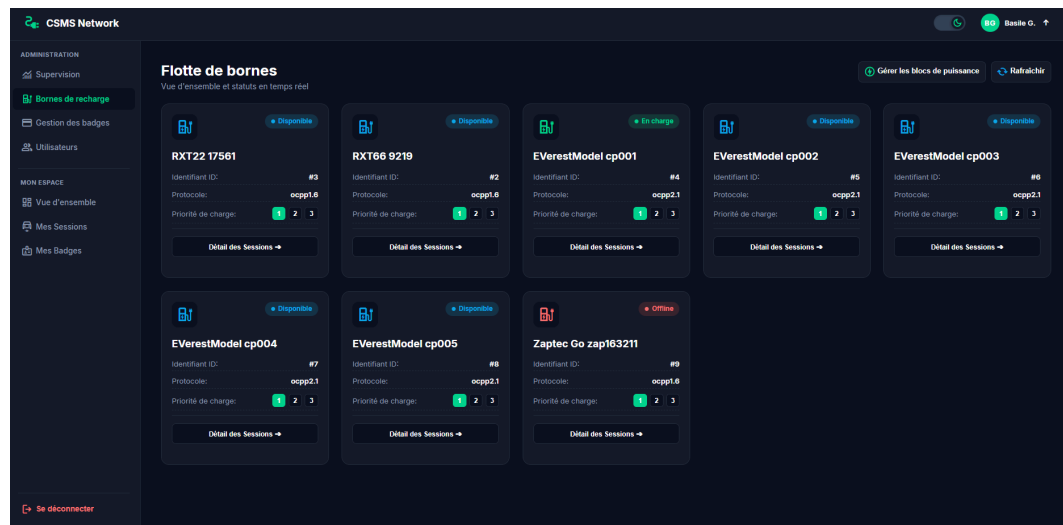


Figure 8: Overview of the Layout (Topbar + Sidebar) and preview of the dark theme.

To ensure instant readability of the fleet's status, a strict semantic color coding was implemented via these global CSS variables:

Green (Emerald Green) : Charging station charging or active.

Blue : Charging station available and ready for use.

Red : Charging station offline, in error, or illegal session.

Finally, for the visual representation of complex data (telemetry, power curves, consumption statistics), the open-source library Recharts was integrated. Its components allow for the generation of dynamic, interactive, and fluid charts, integrating perfectly into the React ecosystem.

5.4 Main Views Development

The interface was segmented to meet the specific needs of two types of profiles: standard users and administrators.

5.4.1 User Interface

The view intended for end users was designed to be direct and informative, revolving around three main tabs:

Overview (Dashboard) : This central screen highlights the parking availability. It displays the user's active charging session in real time, or alternatively, the details of their last session. Global metrics complete this view.

My Sessions : A summary table offering the complete history of charging sessions, with filtering capabilities.

My Badges : A management interface allowing the addition, modification, or revocation of RFID or NFC badges associated with the user's profile.

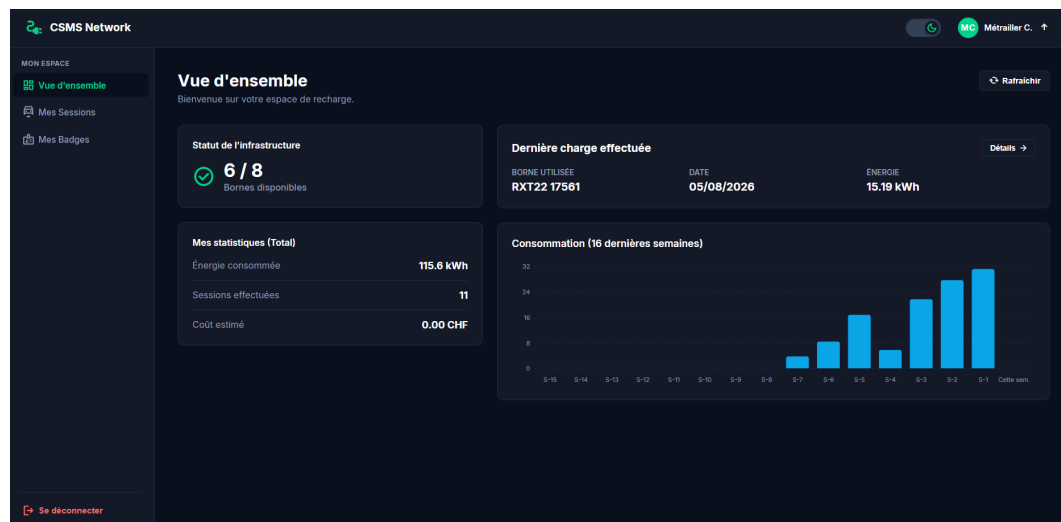


Figure 9: User version of the Dashboard, featuring the overview view and the My Sessions and My Badges tabs.

5.4.2 Administrator Interface

Accounts with elevated rights have access to a set of advanced supervision and configuration tools, distributed across several views:

Global Supervision : The fleet's control center. It consolidates the status of all chargers, the instantaneous power drawn by the system, and the total energy distributed. A graph based on a 24-hour sliding time window traces the precise evolution of the overall load, complemented by an exhaustive list of active sessions.

Charging Stations : The hardware configuration interface. It allows for listing the stations, defining their priority levels, and accessing the management of power blocks to configure the electrical limits of the power supply groups.

Badge Management : An extended view allowing the supervision of all system badges (recognized and unknown) and the auditing of sessions linked to a specific identifier.

Users : The account administration panel, allowing for the modulation of permissions and the inspection of transaction volumes per collaborator.



Figure 10: Administrator version of the Dashboard, featuring the global supervision view and the Charging Stations, Badge Management, and Users tabs.

5.4.3 Detailed Views and Profile

Regardless of the authorization level, clicking on a transaction (from any view) opens a detailed analysis screen of the charging session. This screen renders the temporal curve of the power drawn via an interactive chart and consolidates key indicators. For administrators, this view is enriched with a technical activity log listing the station's events, a diagnostic capability made possible by the richness of the OCPP protocol frames.

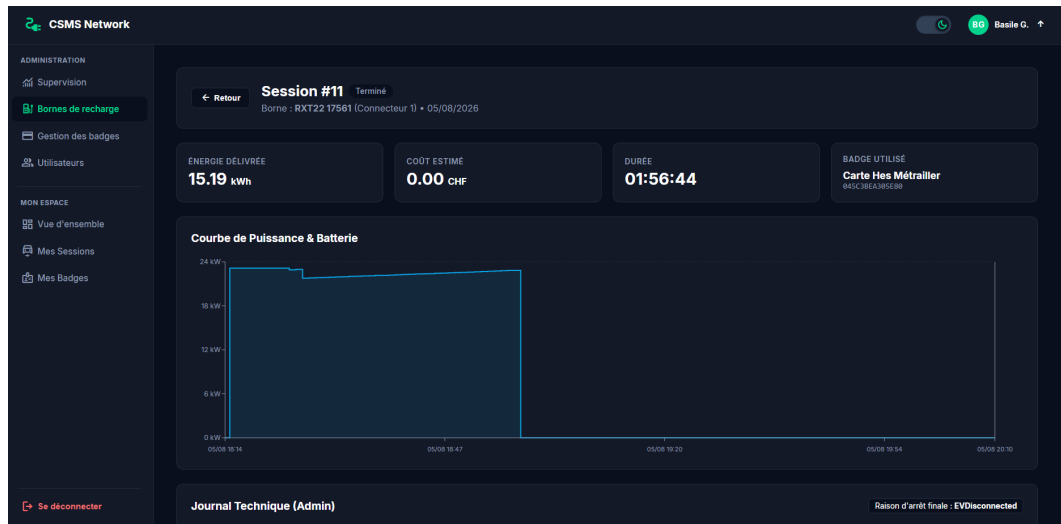


Figure 11: Detailed view of a charging session (with the power variation curve).

Finally, a profile configuration section is available for each account, ensuring user autonomy in managing their personal data and login credentials.

6 | Testing, Validation and Results

6.1 Test Methodology and Environments

System validation was performed across three distinct environments to ensure comprehensive coverage of use cases.

The first environment, purely simulated, relies on five virtual EVERest stations. This framework allowed for the configuration of strict constraints, such as a power block artificially limited to 22A, to force and observe the saturation behaviors of the load-balancing algorithm. The Node-RED code of these simulators was also modified to allow the injection of various RFID identifiers. The procedure to modify the Node-RED code can be found in Appendix AA.

The second environment is a hardware test bench consisting of a Zaptec Go station coupled with a Metrel electric vehicle simulator. This equipment made it possible to electrically manipulate the states of the station and simulate charging sessions without requiring the presence of an actual vehicle, thereby validating the system's responsiveness to real hardware events.

The third environment corresponds to the production infrastructure, composed of the GreenMotion chargers in the HES-SO parking lot. Tests on this hardware were conducted in a targeted manner, focusing on basic functionalities so as not to disrupt service availability for daily users.

6.2 Smart Charging Validation (Load-Balancing)

In order to evaluate the behavior of the dynamic distribution algorithm, the tests were divided into two distinct scenarios. The curves presented below were generated from the raw execution logs of the Smart Charging service, excerpts of which are available in Appendix AB.

The following chronograms illustrate the logical sequence of events. The x-axis represents event triggers, such as vehicle plug-ins, charging stops, or the detection of consumption variations, rather than a strictly proportional time scale. In the background, the service executes a regulation loop clocked at a fixed interval of 30 seconds to evaluate the state of the network.

6.2.1 Scenario A: Saturation and Prioritization

This first scenario validates the management of the maximum limit of a power supply group. On a block limited to 22A, three stations are activated successively. The system allocates the available power to them, favoring the station configured with the highest priority. When a fourth station requests a charge, the algorithm detects that the vital minimum of 6A can no longer be guaranteed for everyone, as the required sum exceeds the 22A limit. The fourth station is therefore placed on standby. Once one of the three active stations is manually deactivated, the released current allows the fourth station to automatically start its session.

For stations under the OCPP 1.6J protocol, sending charging profiles respects a minimum delay of 5 minutes between two consecutive instructions to avoid degrading the physical contactors; this constraint is ignored only during a 0A cut-off.

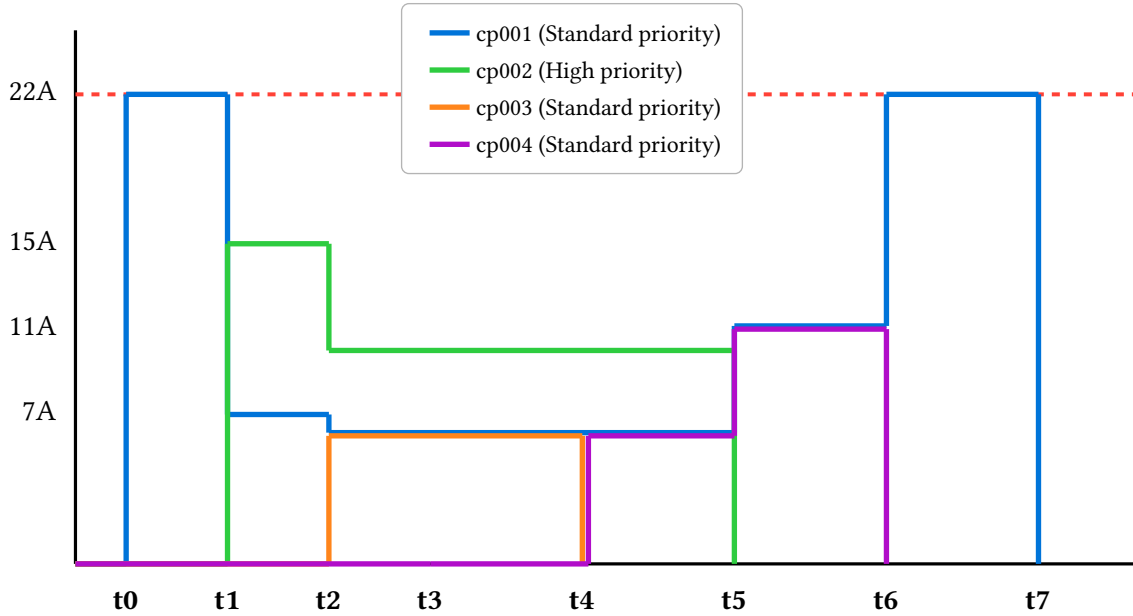


Figure 12: Chronogram of topological changes and weight-based distribution (Scenario A).

- **t0**: Activation of cp001. Being the only one, it receives the maximum group allocation (22A).
- **t1**: Activation of cp002 (with a priority weight of 2). The load is distributed: cp001 drops to 7A, and cp002 takes 15A.
- **t2**: Activation of cp003 (weight of 1). New total balancing divided according to weights (cp001: 6A, cp002: 10A, cp003: 6A).
- **t3**: Activation of cp004 (weight of 1). Since the maximum capacity of 22A is fully distributed, it is placed in the queue (0A).
- **t4**: Deactivation (stop) of cp003. Its session ends, which frees up current and allows cp004 to exit the queue by recovering 6A.
- **t5**: Deactivation (stop) of cp002. Load-balancer adjustment: the free power is distributed equally between the remaining active stations, namely cp001 (11A) and cp004 (11A).
- **t6**: Deactivation (stop) of cp004. cp001, once again alone, recovers the total power (22A).
- **t7**: Deactivation (stop) of cp001. End of all charging sessions on the group.

6.2.2 Scenario B: Under-consumption and Redistribution

The second scenario focuses on optimizing unused power. Two stations are initially active and share the available power. The consumption of one of the simulators is then manually reduced using its web interface to simulate an end of charge or a vehicle reaching its internal hardware limit.

To ensure that telemetric measurements are stabilized after a new instruction is transmitted, the algorithm applies a delay before validating an under-consumption state. This observation delay is set to 90 seconds for equipment operating under OCPP 2.0.1 and

above, and to 210 seconds for equipment under OCPP 1.6. Once this delay has elapsed, the algorithm lowers the limit allocated to the under-consuming station and immediately redistributes the virtually recovered amperes to the other active stations. The system continues to monitor consumption during its 30-second loops and dynamically readjusts the limits as demand increases again.

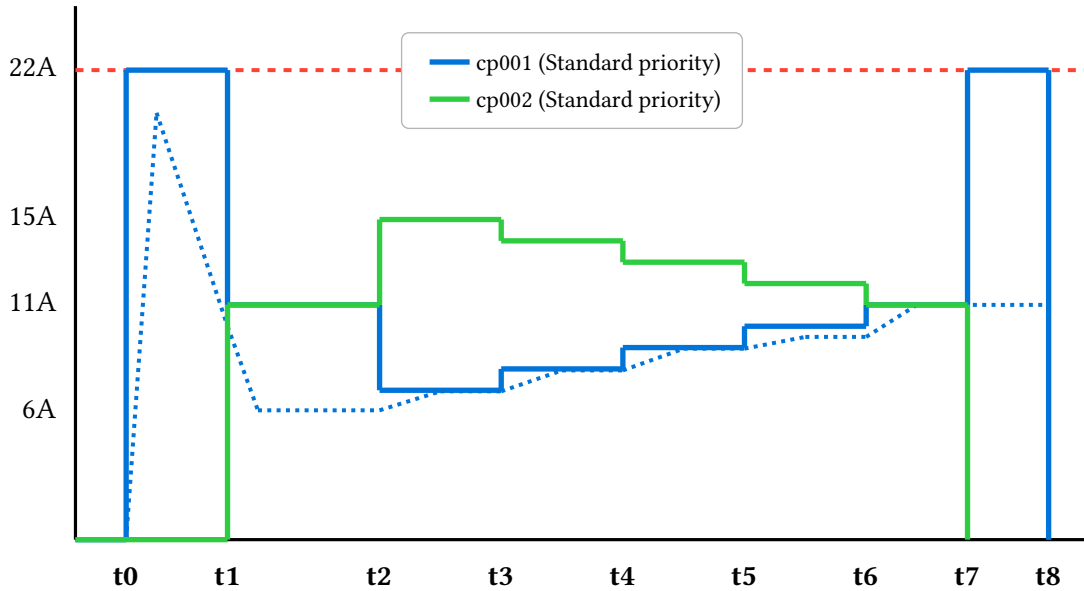


Figure 13: Optimization chronogram by under-consumption detection (Scenario B). The blue dotted line represents the actual consumption of cp001.

- **t0**: Activation of cp001 (22A allocated as it is alone).
- **t1**: Activation of cp002. Default balancing distributes power to 11A for each station.
- **t2**: The algorithm detects significant under-consumption from cp001 (which draws only about 6A despite its authorized 11A). The limit allocated to cp001 is lowered to 7A, and the surplus (4A) is automatically reassigned to cp002 (which goes to 15A).
- **t3 to t5**: The current demand of cp001 gradually increases. At regular intervals, the algorithm observes this increase and restores its capacity in successive steps (8A, then 9A, then 10A) by dynamically subtracting it from cp002.
- **t6**: The consumption of cp001 has increased enough to require its initial 11A again. Both stations are rebalanced.
- **t7**: Deactivation (stop) of cp002. cp001 is once again alone on the network and regains the block's maximum limit (22A).
- **t8**: Deactivation (stop) of cp001. End of the charging scenario.

6.3 Business Flow and Frontend Validation

The validation of business flows confirms that physical events are correctly processed by the software infrastructure and rendered in real time on the user interface.

Access management was tested by scanning various unregistered RFID cards on the EVerest simulators as well as on the Zaptec station. The system correctly intercepted the

authorization denials. These badges instantly appeared in the platform's unknown badges management menu, proving the proper functioning of identifier retrieval within OCPP messages.

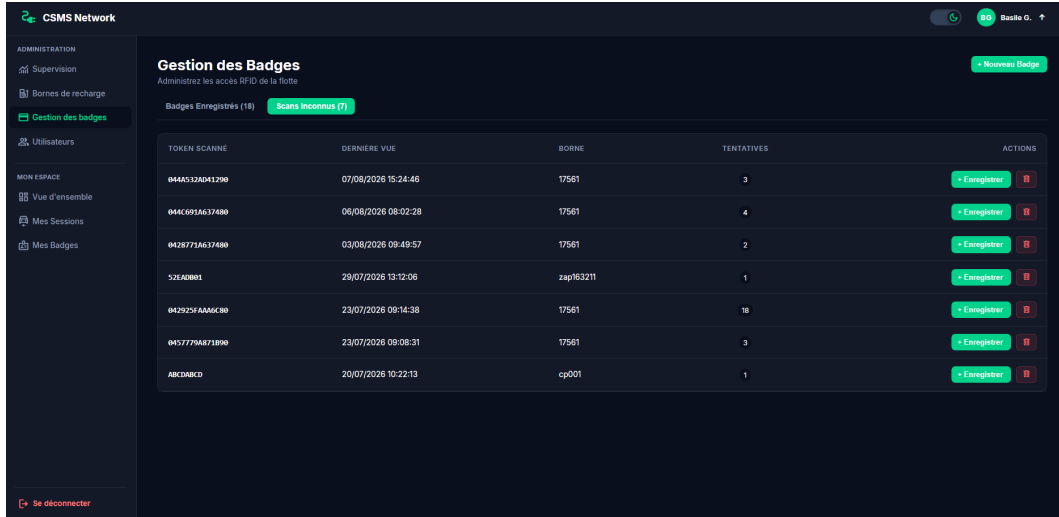


Figure 14: Dashboard showing the menu for adding unknown badges.

The detection of inactive sessions was validated using the Zaptec and Metrel environment and with the help of the Everest simulators. A session was initiated with zero consumption. After the tolerance delay, configured to 20 minutes for this test, the service correctly toggled the transaction status, triggering the appearance of a visual illegality indicator on the supervision dashboard. This behavior was subsequently confirmed organically on the parking lot stations, when users forgot to unplug.

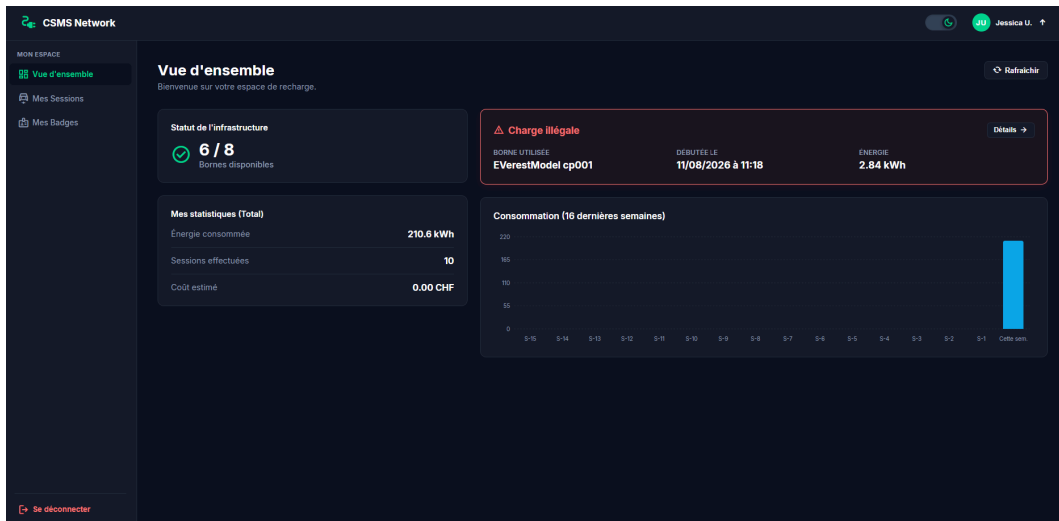


Figure 15: Dashboard showing the visual indicator of an inactive or illegal session.

These various critical events also made it possible to confirm the reliability of the alerting system, with each anomaly successfully triggering the notification routines planned in the centralized architecture. In order to verify and supervise these alerts, a personal email

address was blind carbon copied on all notifications, making it possible to confirm the proper receipt of alert messages by the school staff.

6.4 Hardware Validation

The integration of the physical GreenMotion stations confirmed the hardware connectivity and the proper management of the OCPP 1.6J protocol by the central server. Fundamental operations such as starting a session, authentication, and stopping a charge were successfully validated on this equipment.

However, tests of intensive dynamic variations related to Smart Charging were deliberately avoided on these specific stations. Since periods of simultaneous use of these chargers are rare, and the hardware had demonstrated a certain sensitivity to instruction changes, it was deemed preferable to guarantee the absolute stability of the infrastructure for the school staff rather than risk contactor errors in a production environment.

6.5 Results Analysis (Objectives Verification)

The overall analysis of the tests confirms that the developed system meets the requirements set by the initial specifications. To objectively evaluate the success of the project, the obtained results are directly mapped to the mandatory and optional objectives defined at the beginning of the mandate.

6.5.1 Validation of Main Objectives (Must-have)

- Analysis and state of the art: The comparative study of charging modes and communication protocols was successfully conducted, confirming the relevance of using the OCPP standard.
- Technical infrastructure audit: The Campus charger fleet was fully audited, highlighting the hardware and network limitations of the original installation.
- Management platform analysis: Several open-source solutions were evaluated, leading to the strategic choice of CitrineOS for its modular architecture.
- Definition of OCPP architecture and infrastructure: The centralized system was successfully deployed. It simultaneously manages multiple clients, integrates access management, and ensures secure user authentication.
- Development of a supervision Dashboard: A comprehensive web interface was created. It provides real-time consumption tracking, user history, an administration interface, and NFC identifier management.
- Report writing and technical documentation: The production of this document, coupled with code documentation, validates this reporting requirement.

6.5.2 Validation of Optional Objectives (Nice to have)

- Load distribution algorithm: This objective was fully met. A dynamic load-balancing algorithm was designed, implemented, and successfully validated in a simulation environment, enabling the optimization of the electrical power available on site.
- Retrofitting old charging stations: Due to the technical complexity related to converting OCPP 1.4 to Modbus TCP, this step was not completed during this work and was documented as an area for improvement.

- Advanced billing management: This feature was not prioritized over the development of Smart Charging and remains a future evolution path for integration with the school's administrative systems.

6.5.3 Additional Features

In addition to the specifications, several features not initially planned were developed during the project. The traceability of unknown badges, the detection of inactive sessions, and the modular configuration of power blocks were successfully implemented. These additions provide significant operational value to this proof of concept, making it ready for concrete and future operation.

7 | Conclusion and Future Work

7.1 PoC Assessment

The objective of this diploma project was the design and deployment of a management and supervision platform for a fleet of electric vehicle charging stations. At the end of this work, all the mandatory objectives set by the specifications were successfully achieved. The centralized system, built around CitrineOS and the OCPP protocol, is fully functional. It ensures bidirectional communication with the chargers, real-time consumption tracking via a custom dashboard, as well as the complete management of users and their RFID access.

Beyond the fundamental requirements, the major optional objective consisting of developing a dynamic power distribution algorithm (Smart Charging) was fully designed, implemented within a dedicated microservice, and validated in a simulation environment. This algorithm specifically solved the power limitation problem mentioned in the Section 2.4. This Proof of Concept (PoC) demonstrates that it is technically viable to modernize the campus charging infrastructure while eliminating any dependence on a specific manufacturer.

7.2 Challenges Encountered

The realization of this software architecture required significant upskilling in several modern technologies that had not been covered in the curriculum until then. Developing a comprehensive web interface with React, containerizing application services, and setting up continuous integration pipelines represented a steep but extremely formative learning curve.

On a technical level, configuring the simulation environment proved to be particularly complex. Deploying the five EVerest simulators posed challenges in terms of internal network routing under Docker, due to sometimes sparse documentation on these specific use cases. Furthermore, it was necessary to perform reverse engineering operations and modify the Node-RED source code of the simulators to unlock certain visual aspects of the original interface and allow the manual injection of custom NFC identifiers for the test scenarios.

7.3 Future Work

Although functional, this platform constitutes a technical foundation that needs to be enriched before large-scale deployment.

From a hardware perspective, the integration of the older AC stations on campus remains to be finalized. Since the OCPP 1.4 protocol of these devices is too far removed from current standards, the preferred solution will be to develop a software translation gateway controlling the stations via their Modbus TCP registers, requiring a complete mapping of hardware commands to modern OCPP requests.

Regarding the user experience, the dashboard interface will need to be adapted for mobile devices. This update will allow QR codes to be placed directly on the physical stations, offering users an instant redirection to their charging session tracking portal. In parallel, to guarantee the stability of this interface during future updates, the implementation of automated end-to-end testing must be integrated into the development process.

The network and server infrastructure also require several evolutions. Currently containerized, the overall configuration of the images must be generalized via environment variables to facilitate deployment to other sites. The continuous integration pipeline will need to be complemented by a continuous deployment system automating the updating of services in production. Regarding security, although perimeter protection is already solidly ensured by the Traefik reverse proxy, which manages TLS encryption and strict request routing, the server is now exposed to the internet. Future work should therefore focus on application security, notably through the implementation of a Web Application Firewall or intrusion detection tools to block malicious requests and brute-force attacks on the login interface.

Finally, the business logic will need to align with the institution's standards. The current account creation system is intended to be replaced by a centralized authentication communicating with the HES-SO identity servers. Depending on the school's future administrative decisions, an automated billing module could also be grafted onto the event-driven architecture already in place.

Glossary

Concepts

Smart Charging – Load-Balancing / Smart Charging: Dynamic electrical power distribution algorithm used to optimize consumption without exceeding network limits.

Webhook – Event Trigger / Webhook: Method for real-time data transmission between applications via event-driven HTTP requests.

General

PoC – Proof of Concept: Proof of Concept. A concrete and preliminary experimental realization illustrating a certain method or idea to demonstrate its feasibility.

Hardware

RFID / NFC – Radio Frequency Identification / Near Field Communication: Short-range wireless communication technologies used in this project for the physical authentication of users at charging stations.

Programming

Goroutine – Goroutine: Concurrent function in the Go programming language with a very low memory and CPU footprint.

Protocols

GraphQL – GraphQL: Query language for APIs that allows the client to request exactly the data it needs, utilized here via the Hasura engine.

OCPP – Open Charge Point Protocol: Standard communication protocol between charging stations and a centralized management system.

Security

JWT – JSON Web Token: Open standard for the secure and autonomous exchange of information as a JSON object.

Systems

CSMS – Charging Station Management System: Centralized software system for supervising, managing, and controlling a network of charging stations.

University

HEI – Haute École d'Ingénierie

IT – Infotronics

SYND – Systems Engineering

Bibliography

- [1] “Private One 22 kW - Technical Datasheet.” 2022. [Online]. Available: <https://www.eaton.com/content/dam/eaton/products/emobility/ev-charging/switzerland-ch/xchargein-mobility-ch/PrivateOne-TechnicalDocumentation-ch-fr.pdf>
- [2] “Resilient DC Fast Charger Range XT22 - Specification Sheet.” 2022. [Online]. Available: <https://www.eaton.com/ch/en-gb/catalog/ev-charging/green-motion-dc-22.html>
- [3] “Resilient DC Fast Charger Range XT66 - Specification Sheet.” 2022. [Online]. Available: <https://www.eaton.com/ch/en-gb/catalog/ev-charging/green-motion-dc-44-66.html#resources>
- [4] “Electric vehicle conductive charging system - Part 1: General requirements,” no. IEC 61851-1:2017. 2017.
- [5] “Road vehicles - Vehicle to grid communication interface,” no. ISO 15118-2:2014. 2014.
- [6] “Electromobility - Digital communication between a d.c. EV charging station and an electric vehicle for control of d.c. charging in the Combined Charging System,” no. DIN SPEC 70121. 2014.
- [7] Open Charge Alliance, “Open Charge Point Protocol 1.6 - Specification,” technical report, 2015. [Online]. Available: <https://openchargealliance.org/my-oca/ocpp/>
- [8] Open Charge Alliance, “Open Charge Point Protocol 2.0.1 - Specification,” technical report, 2020. [Online]. Available: <https://openchargealliance.org/my-oca/ocpp/>
- [9] RWTH Aachen University, “SteVe - Open Source OCPP Station Database/Management System.” [Online]. Available: <https://github.com/steve-mws/steve>
- [10] Thoughtworks, “MaEVe CSMS - Open Source Charging Station Management System.” [Online]. Available: <https://github.com/ThoughtWorksInc/MaEVe-CSMS>
- [11] S47 Labs, “CitrineOS Documentation - Open Source CSMS.” [Online]. Available: <https://citrineos.github.io/>

A | Appendix

AA Docker Deployment Guide and Commands

System Deployment and User Guide

This user guide provides detailed instructions for the deployment and use of the electric vehicle charging station management system. It is intended for the personnel responsible for the maintenance and operation of the system.

File Description

The directory containing the entire project is available on the GitHub repository at this link: [https://github.com/Gasile/TB_CSMS]. This repository is structured as follows:

- **.github**: Contains the GitHub Actions workflows for continuous integration.
- **citrineos-config**: Contains the configuration files for the CitrineOS CSMS, allowing the construction of the citrine package image. It also contains the database modification data corresponding to the additions made during development.
- **citrineos-core**: Contains the source code of the CitrineOS CSMS. This folder was cloned from the official CitrineOS repository. The only modifications made are located in the **citrineos-core/apps/Server/everest** folder. This folder contains the simulation environment for the 5 virtual EVerest charging stations. Usage details for these simulators can be found in the section “Setting up the Simulators”.
- **frontend**: Contains the source code of the frontend developed in React. This folder also contains the configuration files for building the frontend package image.
- **services**: Contains the subfolders for each microservice developed in Go. Each subfolder contains the source code of the microservice as well as the configuration files for building the corresponding package image.

At the root of the project, there are also configuration files such as environment files, the .gitignore, and two versions of the docker-compose file.

The **docker-compose-deploy.yml** file is used for deployment using Docker images. This file operates independently. As specified in the report, the proper functioning of this version could not be ensured.

The **docker-compose.yml** file is also used for the complete launch of the project. However, this file requires a full clone of the repository and does not use the Docker images.

Launching the Project

Launching with Docker Images

1. Retrieve the **docker-compose-deploy.yml** file and the **.env.example** file from the GitHub repository.
2. Rename the **docker-compose-deploy.yml** file to **docker-compose.yml** and the **.env.example** file to **.env**, then fill in the environment variables.
3. Place the files in a working directory.
4. Open a terminal and navigate to this directory.
5. Pull the required Docker images by executing the following command:

```
1 docker-compose pull
```

6. Launch the project by executing the following command:

```
1 docker-compose up
```

Launching with the Complete Repository

This method is the most reliable. It is the one used throughout the development of this project. It is also the one currently running on the HEI server.

1. Clone the complete GitHub repository using the following command:

```
1 git clone https://github.com/Gasile/TB_CSMS
```

2. Navigate to the cloned directory:

```
1 cd TB_CSMS
```

3. Rename the **.env.example** file to **.env** and fill in the environment variables.
4. Launch the project by executing the following command:

```
1 docker-compose up
```

It may happen that the citrine container attempts to start before the containers it depends on, which can cause a failure. In this case, simply rerun the **docker-compose up** command until the container starts correctly.

System Usage

Once the system is launched, several web interfaces are available to perform various actions. These interfaces are accessible via a web browser using the server's IP address and the corresponding port for each interface.

Hasura GraphQL Engine

Access to the Hasura GraphQL Engine web interface is available on port 8090. This interface allows interaction with the database via GraphQL queries. This makes it possible to directly supervise the data tables to verify the proper functioning of the system. It is also possible to execute queries to retrieve specific data or to make modifications to

the database. To access it, you must provide the `HASURA_GRAPHQL_ADMIN_SECRET` specified in the `.env` file during project launch.

CitrineOS Central System API

Access to the CitrineOS Central System API web interface is available on port 8080. This interface allows interaction with the CitrineOS CSMS via HTTP requests. This enables direct supervision of the charging stations to verify the proper functioning of the system. It is possible to send requests to the charging stations, allowing for actions such as restarting a station or requesting its startup message (BootNotification).

Frontend Dashboard

The frontend is accessible via the configurations established beforehand during its development. In its current state, it is accessible at the address `evse.hevs.ch`. This configuration is contained in the `docker-compose.yml` file. In the environment variables contained within the frontend image, it is possible to enable an option that activates user and administrator login shortcuts. This avoids having to enter an email and password at each connection. However, it is crucial not to activate this option if the deployment environment is exposed to the internet.

System Update

Microservices, Frontend, and CitrineOS CSMS

As previously mentioned, GitHub Actions have been configured to allow the automatic building of Docker images for each microservice, the frontend, and the CitrineOS CSMS. These actions are triggered every time a modification is pushed to the GitHub repository. This ensures that the Docker images are always up to date with the source code.

If the `docker-compose.yml` used is the version without the images, once the Go code of a microservice or the React code of the frontend is modified, simply run the following command:

```
1 docker compose up -d --build <container_name>
```

Database Modification

As explained above, the **citrineos-config** folder contains the database modification data according to what was added during development. These modifications are carried out via SQL scripts that are executed when the citrine container is launched. This ensures that the database is always up to date with the latest modifications made to the system.

To simplify database modification, the Hasura web interface provides tools to do this in a “graphical” way. In order to save the modifications made via this interface, it is possible to launch a “console” that will listen to and record the modifications made, and add them to the **citrineos-config** folder.

Before starting this procedure, you must clone the git repository to a local directory and add the `hasura.exe` executable to the `citrineos-config/hasura` folder. This executable can be found in the folder submitted for archiving, which contains all the project files. It can also be downloaded online.

1. Open an SSH tunnel using this command:

```
1 ssh -L 8090:localhost:8090 basile.gasser@vlenpchgman.hevs.ch
```

Use valid credentials that have SSH access permissions to the server here.

2. Open a terminal in the TB_CSMS/citrineos-config/hasura folder.
3. Run the following command:

```
1 ./hasura.exe console --admin-secret <HASURA_GRAPHQL_ADMIN_SECRET>
```

Once the command is entered, a window should open automatically in a browser at <http://localhost:8090/console>

4. Make the desired modifications.

If the modifications concern table or column changes, it is important to check the “This is a migration” box.

5. In the terminal, stop the execution by pressing Ctrl+C. After this, the modifications will have been added locally to the citrineos-config folder.
6. Version and push the generated migration and metadata files.

```
1 git add citrineos-config/hasura/
2 git commit -m "migration(db): "
3 git push
```

Setting up the Simulators

In the citrineos-core/apps/Server/everest folder, there is another docker-compose.yml file. Simply run the following command:

```
1 docker compose up -d
```

Each simulator has 3 containers. To modify the number of simulators, simply comment out the 3 containers of each simulator that should be removed in the docker-compose.yml.

Adding an RFID Badge

It is possible to access the Node-RED interface of the simulators via the port indicated under the nodered-X container of a simulator. In the RFID tab on the is_token block, it is possible to add new badges to the “Options” list. The ID must absolutely consist of hexadecimal characters and be 8 or 16 characters in length, otherwise CitrineOS will not recognize the badge. Once the badges are added, go to the menu in the top right, then to Export. In the window that opens, below “Export,” select “all flows” and then the JSON format. Next, download the file and replace the shared-flow.json file in the everest folder with the newly downloaded file.

It is also possible to directly modify the badge list in the JSON file around line 1170.

Once this file has been modified, simply restart the nodered container of the simulators with this command:

```
1 docker compose restart nodered-1 nodered-2 nodered-3 nodered-4 nodered-5
```

Used port List

Port	Service	Description
80, 443	Reverse Proxy (Traefik)	External entry point of the system. Port 80 forces redirection to HTTPS.[cite: 1] Port 443 handles encrypted traffic and dynamically routes it to web interfaces and APIs based on the requested path.
8081	CitrineOS	Main server entry point for communication with external charging stations.
8080	CitrineOS	Port providing access to the internal CitrineOS management API, including the documentation interface accessible via the /docs route.
8090	GraphQL Engine (Hasura)	Direct mapping allowing access to the database engine from the host machine (for administration console or maintenance) without passing through the public reverse proxy.
8086	Auth Service	Restricted access to the host's loopback interface (127.0.0.1), allowing secure internal debugging or administrative operations.

Table 3: Exposed ports on the remote server and their description

AB Raw extracts from the Smart Charging logs

```

1 Scenario A
2
3 smart-charging-service-1 | 2026/08/11 08:32:40 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
4 smart-charging-service-1 | 2026/08/11 08:32:40 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: ccaf52e8-6c4f-40fe-9f99-d94c815bc0e9, Poids: 1) :
  Limite 22.0A
5 smart-charging-service-1 | 2026/08/11 08:32:45 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
6 smart-charging-service-1 | 2026/08/11 08:32:45 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: ccaf52e8-6c4f-40fe-9f99-d94c815bc0e9, Poids: 1) :
  Limite 7.0A
7 smart-charging-service-1 | 2026/08/11 08:32:45 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp002 [ocpp2.1] (Tx: 3ff118e6-530d-4fda-8b94-3c39325bef0e, Poids: 2) :
  Limite 15.0A
8 smart-charging-service-1 | 2026/08/11 08:32:50 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
9 smart-charging-service-1 | 2026/08/11 08:32:50 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: ccaf52e8-6c4f-40fe-9f99-d94c815bc0e9, Poids: 1) :
  Limite 6.0A
10 smart-charging-service-1 | 2026/08/11 08:32:50 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp002 [ocpp2.1] (Tx: 3ff118e6-530d-4fda-8b94-3c39325bef0e, Poids: 2) :
  Limite 10.0A
11 smart-charging-service-1 | 2026/08/11 08:32:50 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp003 [ocpp2.1] (Tx: 22620de1-9071-48b6-933d-740d267a24ba, Poids: 1) :
  Limite 6.0A
12 smart-charging-service-1 | 2026/08/11 08:32:56 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
13 smart-charging-service-1 | 2026/08/11 08:32:56 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp004 [ocpp2.1] (Tx: 0729ed6c-8562-44e3-91e0-0d59f48ba765, Poids: 1) :
  Limite 0.0A
14 smart-charging-service-1 | 2026/08/11 08:33:06 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
15 smart-charging-service-1 | 2026/08/11 08:33:06 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp004 [ocpp2.1] (Tx: 0729ed6c-8562-44e3-91e0-0d59f48ba765, Poids: 1) :
  Limite 6.0A
16 smart-charging-service-1 | 2026/08/11 08:33:16 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
17 smart-charging-service-1 | 2026/08/11 08:33:16 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: ccaf52e8-6c4f-40fe-9f99-d94c815bc0e9, Poids: 1) :
  Limite 11.0A
18 smart-charging-service-1 | 2026/08/11 08:33:16 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp004 [ocpp2.1] (Tx: 0729ed6c-8562-44e3-91e0-0d59f48ba765, Poids: 1) :
  Limite 11.0A
19 smart-charging-service-1 | 2026/08/11 08:33:23 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
20 smart-charging-service-1 | 2026/08/11 08:33:23 ✅ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: ccaf52e8-6c4f-40fe-9f99-d94c815bc0e9, Poids: 1) :
  Limite 22.0A
21 smart-charging-service-1 | 2026/08/11 08:33:30 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
22 smart-charging-service-1 | 2026/08/11 08:33:30 i Power Block 5 : Plus aucune transaction
  active. Arrêt du timer.

```

```

1 Scenario B
2
3 smart-charging-service-1 | 2026/08/11 08:37:55 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...

```

```

4 smart-charging-service-1 | 2026/08/11 08:37:55 ✓ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) :
  Limite 22.0A
5 smart-charging-service-1 | 2026/08/11 08:37:57 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
6 smart-charging-service-1 | 2026/08/11 08:37:57 ✓ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) :
  Limite 11.0A
7 smart-charging-service-1 | 2026/08/11 08:37:57 ✓ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp002 [ocpp2.1] (Tx: dcbd56f9-4a6b-44cb-8677-db565bbe7d90, Poids: 1) :
  Limite 11.0A
8 smart-charging-service-1 | 2026/08/11 08:39:27 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 6.06A | Limite Act: 11.0A | Nvlle Limite: 7.0A
9 smart-charging-service-1 | 2026/08/11 08:39:28 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) : Limite 7.0A
10 smart-charging-service-1 | 2026/08/11 08:39:28 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp002 [ocpp2.1] (Tx: dcbd56f9-4a6b-44cb-8677-db565bbe7d90, Poids: 1) : Limite 15.0A
11 smart-charging-service-1 | 2026/08/11 08:39:57 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 6.95A | Limite Act: 7.0A | Nvlle Limite: 8.0A
12 smart-charging-service-1 | 2026/08/11 08:39:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) : Limite 8.0A
13 smart-charging-service-1 | 2026/08/11 08:39:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp002 [ocpp2.1] (Tx: dcbd56f9-4a6b-44cb-8677-db565bbe7d90, Poids: 1) : Limite 14.0A
14 smart-charging-service-1 | 2026/08/11 08:40:28 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 6.95A | Limite Act: 8.0A | Nvlle Limite: 8.0A
15 smart-charging-service-1 | 2026/08/11 08:40:57 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 7.93A | Limite Act: 8.0A | Nvlle Limite: 9.0A
16 smart-charging-service-1 | 2026/08/11 08:40:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) : Limite 9.0A
17 smart-charging-service-1 | 2026/08/11 08:40:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp002 [ocpp2.1] (Tx: dcbd56f9-4a6b-44cb-8677-db565bbe7d90, Poids: 1) : Limite 13.0A
18 smart-charging-service-1 | 2026/08/11 08:41:28 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 7.93A | Limite Act: 9.0A | Nvlle Limite: 9.0A
19 smart-charging-service-1 | 2026/08/11 08:41:58 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 8.95A | Limite Act: 9.0A | Nvlle Limite: 10.0A
20 smart-charging-service-1 | 2026/08/11 08:41:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) : Limite 10.0A
21 smart-charging-service-1 | 2026/08/11 08:41:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp002 [ocpp2.1] (Tx: dcbd56f9-4a6b-44cb-8677-db565bbe7d90, Poids: 1) : Limite 12.0A
22 smart-charging-service-1 | 2026/08/11 08:42:28 🚫 [DEBUG UNDERLOADING] Block 5 | cp001
  (Poids 1) en sous-charge. Cons: 8.95A | Limite Act: 10.0A | Nvlle Limite: 10.0A
23 smart-charging-service-1 | 2026/08/11 08:42:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) : Limite 11.0A
24 smart-charging-service-1 | 2026/08/11 08:42:58 ✓ [Ajustement (Timer 30s)] Block 5 | Profil
  envoyé à cp002 [ocpp2.1] (Tx: dcbd56f9-4a6b-44cb-8677-db565bbe7d90, Poids: 1) : Limite 11.0A
25 smart-charging-service-1 | 2026/08/11 08:43:50 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
26 smart-charging-service-1 | 2026/08/11 08:43:50 ✓ [Changement Topologique/Poids] Block 5 |
  Profil envoyé à cp001 [ocpp2.1] (Tx: 1947a1c7-46f2-4fa9-8ae4-ef7729b5d165, Poids: 1) :
  Limite 22.0A
27 smart-charging-service-1 | 2026/08/11 08:43:58 🔄 Changement détecté (Table: Transactions).
  Réinitialisation du Block 5...
28 smart-charging-service-1 | 2026/08/11 08:43:58 i Power Block 5 : Plus aucune transaction
  active. Arrêt du timer.

```

AC Database schema (Entity-Relationship Diagram)

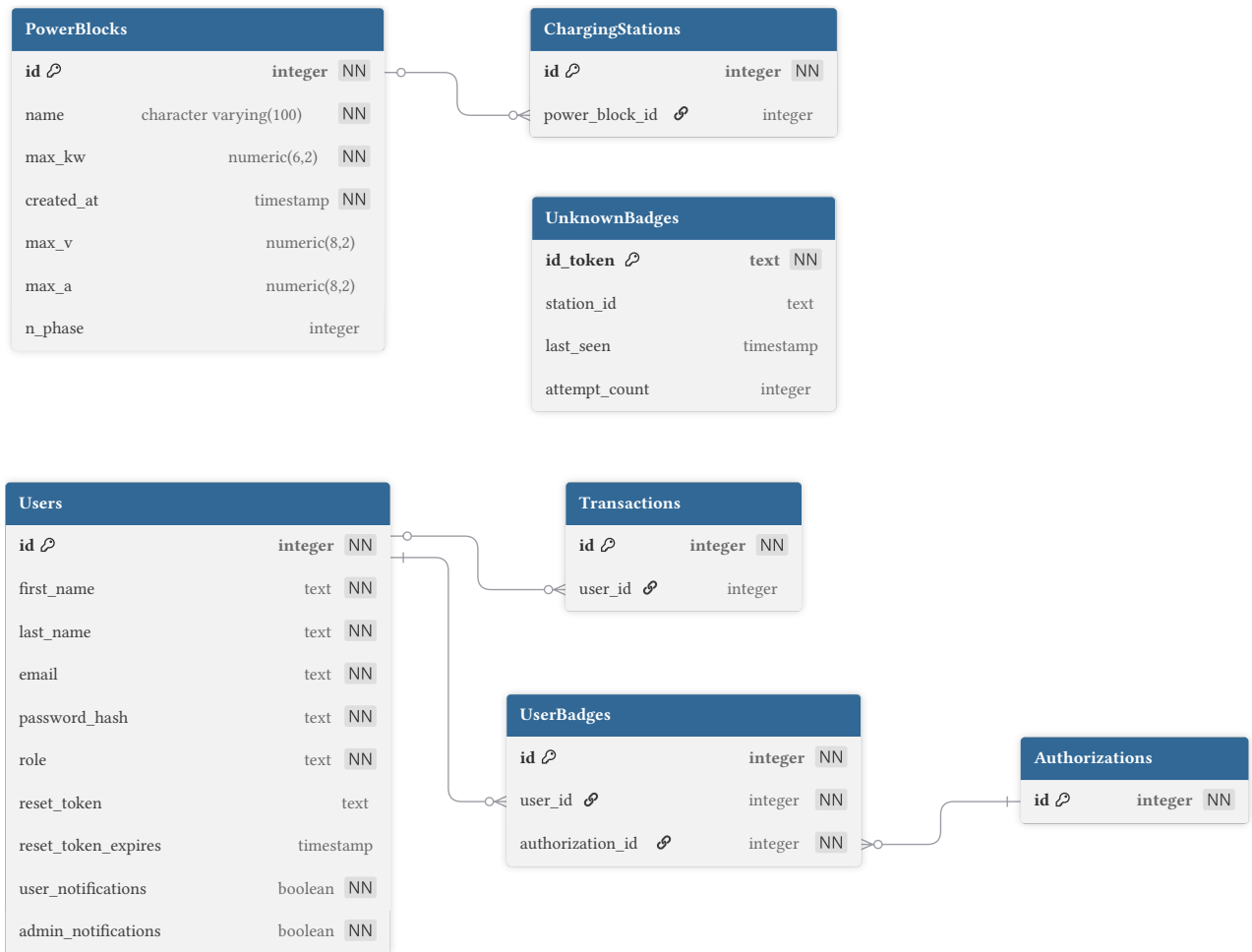


Figure 16: Schéma de la base de données du Smart Charging

AD Code snippets from the Smart Charging algorithm logic

```

1 // =====
2 // APPENDIX: EXTRACT OF THE SMART CHARGING ALGORITHM'S BUSINESS LOGIC
3 // =====
4
5 package smartcharging
6
7 import (
8     "log"
9     "math"
10    "sort"
11    "time"
12 )
13
14 // EVState represents the state of a charging session as used by the algorithm.
15 // This structure groups all the decision-making variables.
16 type EVState struct {
17     TransactionID      string
18     OcppConnectionName string
19     Protocol           string
20     EvseID             int
21     Weight             int // Priority allocated to the vehicle
22     CurrentConsumption float64 // Real consumption reported by the station (MeterValues)
23     AllocatedLimit     float64 // Power limit setpoint calculated by the algorithm
24     Locked             bool // Lock preventing further modification in the same iteration
25     StartTime         time.Time // Start time for FIFO (First-In, First-Out) management
26 }
27
28 // -----
29 // 1. CORE DISTRIBUTION LOGIC
30 // -----
31
32 // calculateLimits determines the power allocated to each vehicle based on
33 // the global capacity of the Power Block (maxA), the session weight,
34 // and the real consumption to optimize available energy.
35 func calculateLimits(powerBlockID int, maxA float64, evs []*EVState) {
36     minAmpsToCharge := 6.0
37     maxActiveCars := int(maxA / minAmpsToCharge)
38
39     // Critical case: Insufficient capacity even for a single car
40     if maxActiveCars == 0 {
41         for _, ev := range evs {
42             ev.AllocatedLimit = 0.0
43             ev.Locked = true
44         }
45         return
46     }
47
48     // Shortage management: if the number of vehicles exceeds the hardware capacity,
49     // we cut off the least priority vehicles (Sorted by weight, then by arrival time).
50     if len(evs) > maxActiveCars {
51         sort.SliceStable(evs, func(i, j int) bool {
52             if evs[i].Weight != evs[j].Weight {
53                 return evs[i].Weight > evs[j].Weight

```

```

54     }
55     return evs[i].StartTime.Before(evs[j].StartTime)
56 })
57
58 for i := maxActiveCars; i < len(evs); i++ {
59     evs[i].AllocatedLimit = 0.0
60     evs[i].Locked = true
61 }
62 }
63
64 var unlockedEVs []*EVState
65 totalOriginalActiveWeight := 0
66 for _, ev := range evs {
67     if !ev.Locked {
68         unlockedEVs = append(unlockedEVs, ev)
69         totalOriginalActiveWeight += ev.Weight
70     }
71 }
72
73 if len(unlockedEVs) == 0 {
74     return
75 }
76
77 // If only one vehicle is active, it gets all available power
78 if len(unlockedEVs) == 1 {
79     unlockedEVs[0].AllocatedLimit = math.Floor(maxA*10) / 10
80     unlockedEVs[0].Locked = true
81     return
82 }
83
84 unallocatedPower := maxA
85
86 // Iteration to detect and redistribute power from vehicles
87 // in "underload" (consuming less than what is allocated to them)
88 for len(unlockedEVs) > 0 {
89     currentIterationWeight := 0
90     for _, ev := range unlockedEVs {
91         currentIterationWeight += ev.Weight
92     }
93
94     if currentIterationWeight == 0 {
95         break
96     }
97
98     if len(unlockedEVs) == 1 {
99         unlockedEVs[0].AllocatedLimit = math.Floor(unallocatedPower)
100         unlockedEVs[0].Locked = true
101         break
102     }
103
104     foundUnderloaded := false
105
106     for i, ev := range unlockedEVs {
107         // Note: lastAppliedLimits and gracePeriodCache are global state variables
108         currentLimit, exists := lastAppliedLimits[ev.TransactionID]

```

```

109     graceEnd := gracePeriodCache[ev.TransactionID]
110
111     baseFairShare := math.Floor(maxA * float64(ev.Weight) /
112     float64(totalOriginalActiveWeight))
113
114     if !exists {
115         currentLimit = baseFairShare
116     }
117
118     // Grace period: allow time for the vehicle to reach its setpoint
119     // before potentially considering it underloaded.
120     if time.Now().Before(graceEnd) {
121         continue
122     }
123
124     isFreshUnderload := ev.CurrentConsumption <= (currentLimit - 2.0)
125     isStillUnderloaded := currentLimit < baseFairShare
126
127     if isFreshUnderload || isStillUnderloaded {
128         neededPower := math.Round(ev.CurrentConsumption) + 1.0
129
130         if neededPower < minAmpsToCharge {
131             neededPower = minAmpsToCharge
132         }
133
134         iterationFairShare := math.Floor(unallocatedPower * float64(ev.Weight) /
135         float64(currentIterationWeight))
136
137         // If the actual need is lower than the theoretical fair share,
138         // we limit the allocation to the actual need to free up the rest.
139         if neededPower < iterationFairShare {
140             ev.AllocatedLimit = neededPower
141             ev.Locked = true
142             unallocatedPower -= neededPower
143
144             unlockedEVs = append(unlockedEVs[:i], unlockedEVs[i+1:]...)
145             foundUnderloaded = true
146             break
147         }
148     }
149
150     if !foundUnderloaded {
151         break
152     }
153
154     // If there are unlocked vehicles and energy left to distribute
155     if len(unlockedEVs) > 0 {
156         distributeWeightedRest(unlockedEVs, unallocatedPower)
157     }
158 }
159
160 // -----
161 // 2. FAIR DISTRIBUTION OF THE REMAINDER
162 // -----

```

```

163
164 // distributeWeightedRest distributes unused energy proportionally to the weight
165 // while guaranteeing the minimum 6A required to charge an EV.
166 func distributeWeightedRest(evs []*EVState, remainingPower float64) {
167     minAmpsToCharge := 6.0
168
169     var unlocked []*EVState
170     for _, ev := range evs {
171         unlocked = append(unlocked, ev)
172     }
173
174     // Phase 1: Guarantee minimum charge (6A)
175     for len(unlocked) > 0 {
176         totalWeight := 0
177         for _, ev := range unlocked {
178             totalWeight += ev.Weight
179         }
180
181         if totalWeight == 0 {
182             break
183         }
184
185         foundUnderMin := false
186         for i, ev := range unlocked {
187             theoreticalShare := remainingPower * float64(ev.Weight) / float64(totalWeight)
188
189             // If the theoretical share is under the minimum, force 6A and recalculate
190             if theoreticalShare < minAmpsToCharge {
191                 ev.AllocatedLimit = minAmpsToCharge
192                 ev.Locked = true
193                 remainingPower -= minAmpsToCharge
194
195                 unlocked = append(unlocked[:i], unlocked[i+1:]...)
196                 foundUnderMin = true
197                 break
198             }
199         }
200
201         if !foundUnderMin {
202             break
203         }
204     }
205
206     // Phase 2: Final distribution proportional to weights
207     if len(unlocked) > 0 {
208         sort.SliceStable(unlocked, func(i, j int) bool {
209             if unlocked[i].Weight != unlocked[j].Weight {
210                 return unlocked[i].Weight > unlocked[j].Weight
211             }
212             return unlocked[i].StartTime.Before(unlocked[j].StartTime)
213         })
214
215         totalWeight := 0
216         for _, ev := range unlocked {
217             totalWeight += ev.Weight

```

```

218 }
219
220 tempPower := remainingPower
221 for _, ev := range unlocked {
222     share := math.Floor(remainingPower * float64(ev.Weight) / float64(totalWeight))
223     ev.AllocatedLimit = share
224     tempPower -= share
225 }
226
227 // Distribution of any remaining amps caused by rounding (math.Floor)
228 for i := 0; i < len(unlocked) && tempPower >= 1.0; i++ {
229     unlocked[i].AllocatedLimit += 1.0
230     tempPower -= 1.0
231 }
232
233 for _, ev := range unlocked {
234     ev.Locked = true
235 }
236 }
237 }
238
239 // -----
240 // 3. HARDWARE PROTECTIONS (Extract from execution function)
241 // -----
242
243 /*
244 Logic block applied just before sending the charging profile to the station.
245 It ensures the preservation of electrical hardware (contactors) and respects
246 network limitations by avoiding spamming of setpoints.
247 */
248 func validateHardwareConstraints(ev *EVState, lastLimit float64, hasLastLimit bool,
249     lastSendTime time.Time, hasLastSendTime bool) bool {
250     // OCPP 1.6 stations are particularly sensitive to frequent variations
251     if ev.Protocol == "ocpp1.6" && hasLastLimit {
252         delta := math.Abs(ev.AllocatedLimit - lastLimit)
253         isCriticalDropToZero := ev.AllocatedLimit == 0.0
254
255         // Rule 1: Minimum delta of 3A (Prevents hardware switching for minor adjustments)
256         // Exception made if the charge must absolutely be cut (DropToZero)
257         if delta < 3.0 && !isCriticalDropToZero {
258             log.Printf("🔴 [OCPP 1.6] Insufficient delta (%.1fA -> %.1fA). Profile ignored.",
259                 lastLimit, ev.AllocatedLimit)
260             return false // Refused to apply
261         }
262
263         // Rule 2: Strict delay (cooldown) of 5 minutes between sending profiles
264         if hasLastSendTime {
265             timeSinceLastSend := time.Since(lastSendTime)
266             if timeSinceLastSend < 5*time.Minute {
267                 return false // Refused to apply
268             }
269         }
270     }
271     return true // Setpoint validated for sending
272 }

```

AE Statement on the Use of Artificial Intelligence

Artificial Intelligence usage declaration

In the context of this bachelor's thesis, Generative Artificial Intelligence tools were utilized in a targeted manner to support research, software engineering, and writing.

Detailed Task Allocation

Gemini Pro 3.1 — Complex Reasoning, Architecture, and Writing

The **Gemini Pro 3.1** model was utilized for phases requiring a comprehensive overview, synthesis skills, and the processing of extended contexts:

- **Literature review and state of the art:** Comparative analysis of standards, technical protocols, and scientific papers.
- **Architecture evaluation and comparative study:** Trade-off analysis (e.g., latency constraints, scalability, architectural choices).
- **Report structuring and outlining:** Logical organization of chapters and subsections to ensure a fluid and rigorous progression of the demonstration.
- **Translation and stylistic revision:** Global translation of the document and harmonization of the academic and technical linguistic register.

Gemini Flash 3.6 — Rapid Prototyping, Debugging, and Typst Diagrams

The **Gemini Flash 3.6** model was reserved for iterative tasks requiring immediate execution:

- **Code module prototyping:** Generation of function skeletons, configuration files, and unit test scripts.
- **Bug resolution and error analysis:** Rapid diagnosis of runtime errors, compilation exceptions, or network request malfunctions.
- **Generation of Typst schemas and diagrams:** Conversion of logical specifications into Typst source code (`#table`, `#grid`, vector components) that can be directly integrated into the report.

GitHub Copilot (VS Code Extension) — Real-Time Inline Assistance

Used daily via the VS Code extension as a contextual typing assistant:

- **Coding assistance:** Autocompletion of code lines, suggestions for repetitive structures, and generation of auxiliary functions.
- **Typst report writing assistance:** Dynamic completion during text, reference, and table/graphic tag entry within the IDE.

AF Email Notification Example



Figure 17: Example of an unknown badge scan notification.

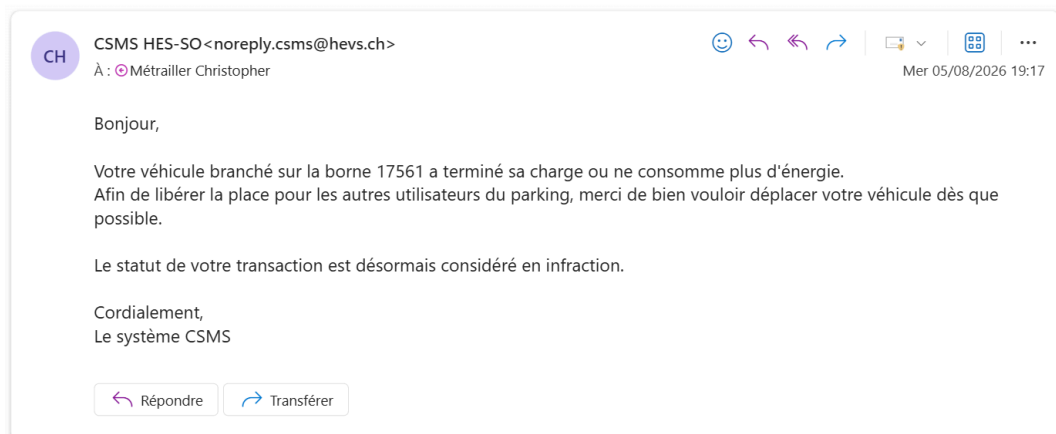


Figure 18: Example of an illegal session detection.



Figure 19: Example of a suspended session due to the lack of available power.